Mitigating Catastrophic Forgetting with Physics-Indexed Specialist Memory: the Evolutionary Neuroplastic Agent

Blas Eugenio Vicco Zarate
Independent Researcher
Valencia, Spain
blasvicco@gmail.com

***Abstract*—Continual reinforcement learning requires adaptation to changing conditions while preserving performance under previously encountered dynamics. It was investigated whether specialists generated by evolutionary search can provide a reusable memory when combined with environment detection and failure-driven search. The Evolutionary Neuroplastic Agent (ENA) evolves small neural networks without gradients, archives selected policies by detected dynamic regime, and retrieves them through Bayesian policy reuse. The architecture was evaluated in non-stationary CartPole and HalfCheetah environments. In the 30-replicate CartPole ablation, the complete architecture achieved a mean success rate of 83.2% across the five test environments. Among variants that learned the harder CartPole environments, the complete architecture showed the most consistent retention. A selection ablation also showed that the retrieval rule affected performance. In the evaluated sequences, ENA adapted more slowly than PPO on CartPole and achieved lower returns than SAC on HalfCheetah. The evidence concerns the complete architecture and does not isolate gradient-free optimization as the cause of retention or establish superiority over other continual-learning methods.**

***Index Terms*—Continual Reinforcement Learning, Neuroevolution, Non-Stationary Environments, Change Detection, Policy Reuse**

I. Introduction

Reinforcement learning (RL) agents are usually trained and deployed on the assumption that the environment does not change. When it does, for instance when a robot's physics shift between conditions, an agent with a single policy adapts by overwriting the parameters that encoded its previous behavior. This is *catastrophic forgetting* [1]. When the original conditions return, the agent has to learn them again.

Continual reinforcement learning studies agents that keep adapting without losing what they have already learned [2]. One family of methods protects or partitions the weights of a shared network, by regularization [3], replay [4] or task-specific heads [5]. Another keeps a separate policy for each detected context and recalls it when that context returns [6]–[8]. Neuroevolution offers a third route. It trains neural networks by mutation and selection over a population of candidates, instead of by gradient descent. Nisioti et al. [9] showed that evolution strategies balance adapting to new tasks and remembering old ones well, while genetic algorithms adapt fastest but forget more. In these methods, however, the whole population evolves toward the current task. The population receives no signal that the environment has changed, nor indication of which earlier solution to restore when an earlier environment returns.

Evolutionary methods learn by trying many candidate solutions at once. Along the way, they produce many solutions that already work in the conditions they were tried in. The question addressed in this paper is whether a small set of skills can turn those solutions into a memory that mitigates forgetting. The skills are inspired by how organisms vary under stress and consolidate memories. The **main research question** is:

RQ1 When an agent encounters a sequence of non-stationary environments without explicit change notifications, does preserving effective policies indexed by detected physical dynamics—combined with failure-driven modulation of search effort—mitigate catastrophic forgetting?

Three narrower questions examine this memory mechanism in detail:

RQ1a Is preserving successful policies sufficient on its own, or must the archive be explicitly indexed by the physical regime under which each policy was validated?

RQ1b Does policy archiving require additional environment interaction, or does it operate as an emergent byproduct of population evaluations?

RQ1c Can dynamic shifts and previously encountered regimes be autonomously detected and identified purely from unassisted agent experience?

Searching incurs computational cost, as candidate policies require evaluation in the environment. The **secondary research question** is therefore:

RQ2 Does dynamically concentrating search effort during performance failure achieve competitive retention and reliability while significantly reducing the computational cost incurred by uniform full-effort search?

To address these questions, the **Evolutionary Neuroplastic Agent (ENA)** is introduced. ENA evolves a population of small neural networks, termed *specialists*. Each specialist is scored with a *Trust* metric tracking recent return history.

Search effort, termed *Plasticity*, scales dynamically with the performance deficit of the active specialist. Validated specialists are archived in a *Hall of Fame*. *Environment detection* identifies dynamic transitions and recognizes previously encountered physical regimes strictly from unassisted agent observations. On CartPole, it evaluates predictive accuracy under transition models of each known regime. On HalfCheetah, it applies a structured diagnostic perturbation—a *chirp*—and analyzes mechanical response features. *Specialist selection* governs which archived specialist to deploy via Bayesian policy reuse [10], integrating detector evidence with observed episodic returns. As a structural design constraint, no component of ENA computes gradients.

Two domains are studied. In CartPole [11], the agent balances a pole on a moving cart. In HalfCheetah [12], a simulated two-legged robot learns to run. In each domain, the agent trains on three environments in turn, which differ in gravity and other physical parameters (Section IV). The evidence for the claim is a *skill ablation*: the same agent, with each skill switched off in turn. Alongside it, standard single-policy learners were trained on the same sequences of environments, with the same number of environment steps: DQN [13], PPO [14] and SAC [15]. They show how much standard learners forget on these streams. They are a reference, not opponents. *It is not claimed that ENA outperforms other continual-RL methods*, nor that gradient-free learning is what prevents forgetting. The question is only whether these skills mitigate it. Comparing ENA with continual-RL systems built for the same problem is left to future work (Section VI-D).

The contributions, each tied to the question it answers, are:

- **A skill ablation** on CartPole, which switches each skill off in turn (RQ1, RQ1a, RQ1b, RQ2).
- **A selection ablation**, which compares Bayesian policy reuse with simpler ways of choosing an archived specialist (RQ1a).
- **A measurement of forgetting** in standard single-policy learners, on CartPole and HalfCheetah, as the reference for RQ1.
- **Gradient-free environment detection** for both domains, including the chirp for HalfCheetah, evaluated on a detection benchmark (RQ1c).
- **A study of what failure-driven search costs**: one run at full effort in every episode, and two alternative rules for turning failure into search effort (RQ2).
- **Reproducibility**: every result and setting reported here is generated by scripts, from the released code and experiment outputs.

Section II reviews related work. Section III states the problem and describes ENA. Section IV presents the environments, protocols and baselines, and Section V the results. Section VI answers the research questions and discusses the limitations. Section VII concludes.

II. Related Work

A. Non-Stationary and Continual Reinforcement Learning

A neural network trained by gradient descent on one task after another degrades on the earlier tasks. This failure was first described as catastrophic interference [1]. Continual reinforcement learning studies agents that must keep learning in such conditions without losing what they already know [2]. Elastic Weight Consolidation [3] slows learning on the weights that mattered for earlier tasks. CLEAR [4] replays past experience with behavioral cloning, and needs no task boundaries. When tasks are incompatible, a single network cannot serve them all. For such tasks, OWL [5] gives each task its own policy head on a shared feature extractor, and chooses a head at test time with a multi-armed bandit. These three methods protect or partition the weights of one gradient-trained network. ENA, like the context-based methods below, preserves whole policies instead. An archived specialist is never trained again, so there is nothing to protect.

B. Evolutionary Reinforcement Learning and Continual Neuroevolution

Neuroevolution optimizes neural networks by mutation and selection over a population, instead of by gradients [16]. Evolution strategies [17] and genetic algorithms [18] are competitive with gradient-based RL on standard benchmarks. They need many samples, however, and optimizing many parameters remains hard for them [19], [20]. Hybrid methods use a population to diversify the experience of a gradient-trained learner [19]. Nisioti et al. [9] compared evolution strategies and genetic algorithms with PPO, continual RL methods and population-based RL, under changes of task and environment. Evolution strategies achieved the best balance between adapting and remembering. Genetic algorithms were the most plastic, but forgot more. The authors related this balance to how much the neighborhoods of good solutions in weight space overlap between consecutive tasks.

Earlier work on evolutionary algorithms for changing optimization problems developed remedies such as increasing diversity after a change, maintaining diversity throughout the run, and memory [21]. ENA uses these three (Section III-C). In the methods Nisioti et al. study, as in standard neuroevolution, the population evolves as a whole toward the current task. Changes are not detected, and earlier solutions are not stored and recalled. ENA adds both without gradients: it identifies the dynamics online, and it preserves specialists indexed by their physics.

C. Quality-Diversity and Repertoire-Based Adaptation

Quality-Diversity algorithms such as MAP-Elites [22] search for many diverse, high-performing solutions instead of a single optimum, and store them in an archive. Intelligent Trial and Error (ITE) [23] computes such an archive offline, in simulation. With it, a damaged robot finds a working gait within minutes, by trying archived behaviors guided by a Bayesian model. TeLAPA [24] brings the same idea to continual RL. For each task, it trains a base policy with PPO

and fills an archive of behaviorally diverse policies around it with MAP-Elites. When a new or revisited task begins, short adaptation probes pick a starting point from all archives, so that competent alternatives remain available when a task is revisited. TeLAPA is evaluated on MiniGrid and on cyclic friction shifts in MuJoCo, HalfCheetah included, with the task boundaries given. ENA's Hall of Fame is also an archive of specialists, but it differs in two ways. It is built online, while the environment changes. It is organized by the physics the agent detects, not by a behavior descriptor.

D. Change Detection and Policy Reuse

Sequential change detection asks when a data stream has changed. Classic methods range from Page's CUSUM [25] to Bayesian online changepoint detection [26]. On CartPole, ENA uses the Predictive-Mixture CuSum [27]. It detects a change away from a known distribution without a model of the distribution after the change, by mixing predictors fitted on recent windows of data.

In RL, RL-CD [6] learned one model of the environment per *context*, and recognized contexts by which model predicts best. ENA follows this idea. MBCD [7] combined an ensemble of learned probabilistic dynamics models with a high-confidence change-point detector whose false-alarm rate is bounded. It created a new policy for each new context, and reused an earlier one when a context recurred. SWOKS [8] detects contexts with optimal-transport distances and Kolmogorov–Smirnov tests, and trains one policy per context. These systems are the closest to ENA in purpose, and they too preserve a dedicated policy per context. RL-CD is tabular, while MBCD and SWOKS train their models and policies by gradient descent.

ENA differs from these systems in how memory is populated and recalled, rather than in representation. Its specialists are individuals generated inherently by population search, rendering policy archiving free of dedicated training overhead. The search that produces them is scaled to failure. Recall weighs the detector's evidence together with the observed returns. Section VI-D explains why these systems are not baselines here. To choose which stored policy to deploy, Bayesian Policy Reuse [10] keeps a belief over task types, updated from the returns it observes. ENA adapts it to an online, non-stationary setting, and adds the dynamics models' likelihoods to the evidence (Section III-I).

E. Positioning

Table I summarizes how related methods preserve and recall earlier behavior. OWL and TeLAPA are told the task boundaries during training, so they detect nothing online. ITE starts adapting when it senses a performance drop. TeLAPA is partly gradient-free: it trains with PPO, and fills its archives by mutation. ENA does not introduce any of these capabilities on its own. Change detection, recurrence recognition, whole-policy preservation, gradient-free learning and probing all have precedents. What ENA combines is threefold. Its archive is populated with candidate solutions generated naturally during evolutionary search (RQ1b). The archive is indexed online by physics the agent detects without labels (RQ1c). The search that feeds it scales its effort to failure (RQ2). No component computes a gradient. Unlike most methods in the table, ENA also assumes that it can reset the environment: to evaluate candidates and, on HalfCheetah, to play its chirp.

III. Methodology

A. Problem Setting

The agent meets a sequence of environments $\mathcal{M}_1, \ldots, \mathcal{M}_K$. Each is a Markov decision process $\mathcal{M}_k = (\mathcal{S}, \mathcal{A}, P_k, r)$. All of them share the state space, the action space and the reward function. They differ only in their transition dynamics P_k, that is, in their physics. During training, the environments are presented one after the other, each for a fixed budget. In the skill ablation, the budget is a number of episodes; when ENA is compared with other learners, it is a number of environment steps (Section IV). The agent observes only states, actions, rewards and episode ends. It is never told when the environment changes, which environment it is in, or how many there are (Appendix A), and it keeps all of its state across the changes. After training, the agent is tested, frozen, on a sequence that contains every training environment and some it has never seen. Again, it is not told where one environment ends and the next begins.

An agent *forgets* environment $\mathcal{M}_k$ to the extent that its test return there falls below its return at the end of $\mathcal{M}_k$'s training phase. Section IV-D measures this as retention and absolute forgetting. An agent *adapts* when it learns each new environment within its budget. The goal is to adapt to every new environment without forgetting the earlier ones. This setting is narrower than continual RL in general [2]: the observation space, the action space and the reward stay the same, and only the physics change.

B. Overview and Terms

The Evolutionary Neuroplastic Agent (ENA) never computes a gradient. It maintains a population of N small neural-network policies, the *specialists*, and each specialist carries a scalar *Trust* score. In every training episode, one specialist acts, and its return updates its Trust. A *Plasticity* value then decides how much of the population is re-evaluated. The acting specialist may be archived in a *Hall of Fame*, and the population evolves. Two more skills make the archive usable when the environment changes. *Environment detection* notices when the physics change and recognizes physics seen before. *Specialist selection* chooses which archived specialist to deploy. Each skill is an interchangeable module with an "off" setting, which makes the ablation of Section V possible. Fig. 1 summarizes one training episode.

The biological names are labels, not claims. In operational terms:

- a *specialist* is one policy network of the population;
- a *gladiator* is a frozen copy of a specialist, stored in the Hall of Fame;

TABLE I
HOW RELATED METHODS KEEP AND RECALL EARLIER BEHAVIOR

Method	Memory	Online detection	Recurrence	Gradient-free	Probing
EWC [3]	regularized shared weights	–	–	–	–
CLEAR [4]	replay into shared weights	–	–	–	–
OWL [5]	shared extractor (EWC), one head per task	–	bandit at test	–	–
RL-CD [6]	model and policy per context	✓	✓	tabular	–
MBCD [7]	model and policy per context	✓	✓	–	–
SWOKS [8]	policy per context	✓	✓	–	–
TeLAPA [24]	per-task archives of initializations	–	–	partly	few-shot probes
Nisioti et al. [9]	population only	–	–	✓	–
ITE [23]	offline behavior archive	performance drop	–	✓	trials
ENA	**frozen specialists per niche**	✓	✓	✓	**chirp (HalfCheetah)**

1: select the acting specialist i (ϵ-greedy on Trust)
2: play one episode, feeding every transition to the detector
3: **if** a change of physics is confirmed **then**
4: start a new regime: reset the detector's evidence, the input statistics and the return history H
5: **end if**
6: $D \leftarrow f(R)$; $T_i \leftarrow (1 - D)\, T_i + D\, \hat{r}(R)$
7: if $D > 0.1$, blend the Trust of every non-archived specialist toward the reference
8: $\mathcal{P} \leftarrow f(R)$
9: Hall of Fame: gate i, resolve its niche, archive or clone (on HalfCheetah, hold an Olympiad when due)
10: evaluate $k(\mathcal{P})$ other specialists for $s(\mathcal{P})$ steps each; update their Trust
11: keep the elites; replace the rest by mutants, crossovers and fresh specialists (gladiators are never replaced)

Fig. 1. One ENA training episode. f is the failure signal (1), T_i the acting specialist's Trust (3), $\mathcal{P}$ the Plasticity, and k, s the number and length of candidate evaluations (4) (Section III-E). At test, the weights are frozen, $\mathcal{P} = 0$, and Bayesian policy reuse chooses the acting specialist (Section III-I).

- a *niche* is a set of physics that the detector treats as one environment, numbered by ENA itself and never given by the experiment;
- *Trust* is a running score of a specialist's recent normalized returns, used to rank specialists;
- *Plasticity* is a number in $[0, 1]$ that sets how much search effort an episode receives.

C. Inspiration

ENA borrows two strategies from nature. Each one works through a mechanism that evolutionary algorithms for changing environments already use.

1) Variation in proportion to stress (Plasticity): Bacteria under stress increase their mutation rate. The response is regulated and transient. Variation rises when the current genotype is failing and stays low otherwise, because most mutations harm a well-adapted organism [28]. Evolutionary computation adopted the idea as *triggered hypermutation*, which raises the mutation rate when performance drops in a non-stationary environment [29]. More generally, it adopted the idea of controlling an algorithm's parameters during the run [30].

ENA's Plasticity extends this response from the mutation rate to the whole search effort. The failure signal (1) sets three things: how many specialists are re-evaluated (a random subset of size k), how long each is evaluated (s steps), and how far mutants move from their parents. The share of the population replaced in each generation stays constant. What scales with failure is the evidence gathered before selection, and the size of the steps taken after it. The rationale is cost. Every candidate step is an environment step, so a converged agent should spend few of them, while an agent that has just met new physics needs to search more broadly. Freshly initialized specialists also enter the population in every generation, as *random immigrants* [31] do. Their purpose is to keep the population diverse enough to follow a moving optimum.

2) Consolidated memory (Hall of Fame): The complementary learning systems theory [32] holds that the brain learns quickly in the hippocampus and consolidates slowly into the neocortex. There, knowledge is protected from interference by new learning. The theory was proposed in part to explain why the brain does not suffer the catastrophic interference of gradient-trained networks [1]. ENA follows the same division of labor. The population is the fast learner, overwritten in every generation. The Hall of Fame is the consolidated store. It is entered only through a gate, it is never modified, and evolution never replaces it.

In evolutionary computation, a *hall of fame* first kept the best individuals of past generations in competitive coevolution [33]. Memory-enhanced evolutionary algorithms store good solutions and reinsert them when earlier conditions return [34]. ENA adds two things to this kind of memory. The memory is indexed by the physics in which each specialist succeeded (Section III-H). Recall is a Bayesian decision rather than a reinsertion (Section III-I).

D. Population and Specialists

Each specialist is a small multi-layer perceptron. On CartPole, its hidden layers have 4×2 ReLU units, with orthogonal initialization. On HalfCheetah, they have 32×16 tanh units, with normc initialization and a tanh output, as in the evolution-strategies policy of Salimans et al. [17]. A specialist's inputs are normalized with a running mean and variance. These statistics keep drifting while the agent explores. Before a specialist acts, its first layer is re-scaled analytically to the current statistics, so that its behavior is preserved without retraining. During training, the acting specialist is chosen

ϵ-greedily. With probability ϵ, it is a random non-archived specialist; otherwise, it is the specialist with the highest Trust.

E. Trust and Plasticity

Trust and Plasticity both derive from the same failure signal. Let H be the history of the last 100 episode returns, and $P_q(H)$ its q-th percentile. Let $R_{\min}$ be the worst return of the current regime. Both H and $R_{\min}$ are cleared when a change of physics is confirmed (Section III-H). For a return R,

$$f(R) = \begin{cases} 1 & |H| < 10 \\ 0.1 & R \geq P_{90}(H) \\ 1 - 0.9\,\rho(R) & \text{otherwise,} \end{cases} \tag{1}$$

$$\rho(R) = \operatorname{clip}\left(\frac{R - R_{\min}}{P_{90}(H) - R_{\min}},\, 0,\, 1 \right). \tag{2}$$

A return at or above the 90th percentile gives the minimum signal, 0.1. Below it, the signal grows linearly, up to 1 at the worst return observed. The function f is termed the *linear ramp*.

1) Trust: The acting specialist's Trust is an exponential average, whose rate is the failure signal $D = f(R)$:

$$\begin{aligned} T_{t+1} &= (1 - D)\, T_t + D\, \hat{r}(R), \\ \hat{r}(R) &= \frac{R - P_5(H)}{P_{95}(H) - P_5(H)}, \end{aligned} \tag{3}$$

with $\hat{r}$ clipped to $[-2, 5]$. The clip is deliberately wider than $[0, 1]$, so that a specialist that beats the current record still ranks above the rest. A poor episode moves Trust quickly, and a good one only nudges it. Most episodes also re-open the competition. Whenever $D > 0.1$, that is, whenever the return falls below the 90th percentile of H, the Trust of every non-archived specialist is blended by the factor D toward a reference value, the normalized mean of H.

2) Plasticity: Plasticity $\mathcal{P} \in [0, 1]$ sets the search effort of an episode, and a *controller* sets it from the episode's return (Section III-E3). Plasticity chooses k of the other $N-1$ specialists as candidates, and evaluates each one for at most s steps:

$$\begin{aligned} k &= \min\left(N - 1,\, \max\left(2,\, \lfloor N\mathcal{P} \rfloor\right)\right), \\ s &= s_{\min} + (s_{\max} - s_{\min})\, \mathcal{P}. \end{aligned} \tag{4}$$

Here $s_{\min} = 10$, and $s_{\max}$ is the full CartPole episode (500 steps) or 200 steps on HalfCheetah. Candidates play in a copy of the current environment, and a truncated return is rescaled to a full-episode equivalent. Their Trust is then updated with (3). Plasticity also scales the mutation noise (Section III-F). In short, a confident agent evaluates 2 candidates briefly and mutates gently, while a failing agent searches broadly and deeply. Every candidate step is a real environment step. When ENA is compared with other learners, these steps count against ENA's budget. In the skill ablation, whose budget is a number of episodes, they are extra interaction.

3) Plasticity controllers: A Plasticity controller maps the return of an episode to $\mathcal{P}$. Trust always uses the linear ramp (1) as its rate, so only the Plasticity controller changes. Five controllers are defined:

- *Linear ramp*, ENA's controller: $\mathcal{P} = f(R)$ (1).
- *Zero*: $\mathcal{P} = 0$ in every episode. This is the minimum search: $k = 2$ candidates for $s_{\min}$ steps each, and the lowest mutation noise. The ablation uses it to switch Plasticity off.
- *Full*: $\mathcal{P} = 1$ in every episode. Every other specialist is evaluated for $s_{\max}$ steps, and mutants receive the largest noise. It is the reference for a fixed maximum effort (Section V-B).
- *Quadratic*:

  $$\mathcal{P} = \operatorname{clip}\left(\left(\frac{R_{\max} - R}{R_{\max} - R_{\min}} \right)^2,\, 0.1,\, 1 \right),$$

  where $R_{\max}$ is the best return of the current regime. It stays low for moderate failures, and rises steeply only near the worst return.
- *Fuzzy*: a Mamdani controller [35] with two inputs. The first input is the normalized return $\rho(R)$. The second is the return's recent trend: the mean of the last 5 returns, minus the mean of up to 45 returns before them. The trend is divided by $P_{90}(H) - R_{\min}$, the span of ρ, and clipped to $[-1, 1]$. It is 0 until 20 returns have been seen. The controller's 9 rules raise Plasticity when the return is low or falling, and lower it when the return is high and steady or rising. Inference is min–max, with centroid defuzzification.

Like the ramp, the quadratic and fuzzy controllers return 1 until 10 returns have been seen, and they stay within $[0.1, 1]$. Section V-C compares them. At test, $\mathcal{P} = 0$ whatever the controller. When environment detection confirms a change of physics, H is cleared. The ramp, quadratic and fuzzy controllers then search at full effort until they have seen 10 returns of the new regime.

F. Evolutionary Search

After every training episode, the specialists are ranked by Trust, and ties go to the older specialist. A specialist's age counts the episodes it has played, and a mutant inherits its parent's age. The top 20% form the elite pool. The bottom 80%, except archived specialists, are replaced. Of the replaced specialists, 40% become mutants of a random elite and 40% become crossovers of two elites. The remaining 20% are freshly initialized, which keeps the population diverse. In all, with no archived specialist among them, each generation replaces 32%, 32% and 16% of the population with mutants, crossovers and fresh specialists.

A mutant perturbs each weight of its parent with probability p_m, by Gaussian noise of standard deviation $0.01 + (\sigma_m - 0.01)\,\mathcal{P}$. A crossover takes each weight from either parent with probability 0.5, after both parents are re-scaled to the current input statistics. New specialists start at the reference Trust.

G. Hall of Fame

The Hall of Fame stores frozen copies of proven specialists, the *gladiators*. It holds at most K gladiators per *niche*, a niche being a set of physics recognized by environment detection (Section III-H). The evolutionary step never replaces a gladiator. After each training episode, the acting specialist is considered for induction, in five steps:

1) *Gate.* Its Trust must stand out from the population, $z = (T - \mu_T)/\max(\sigma_T, 0.1) \geq 1.5\,(1 - 0.5\,\mathcal{P})$, or reach $T \geq 0.5$. High Plasticity lowers the induction threshold during active search.
2) *Niche.* Environment detection resolves which niche the episode was played in.
3) *Cloning, not relabeling.* When a gladiator also succeeds in a different niche, it is cloned into that niche. The original retains its home niche assignment.
4) *Diversity.* A candidate may be a near-copy of a gladiator of the same niche, with a cosine similarity of weights above 0.95. It then replaces that twin only if its Trust is higher. Weight similarity is a cheap but imperfect proxy for behavioral similarity. It catches near-copies, such as a mutant and its parent. Two networks can still behave alike with different weights, for instance with permuted hidden units.
5) *Capacity.* A candidate fills a free slot of its niche, or replaces the niche's weakest gladiator if it is better.

This design decouples policy preservation from ongoing adaptation:

- *Retention.* A gladiator is a frozen copy, and the Trust stored with it at induction does not decay. Only a candidate of its own niche with strictly higher Trust can displace it. Gladiators of other niches can never evict it, however long its own physics stay away. Gladiators in the population are also exempt from the evolutionary replacement and from the Trust blend.
- *Within-niche improvement.* Within a niche, the archive continues to improve. A better candidate takes a free slot or the weakest gladiator's place, and high Plasticity lowers the gate. Success in a new niche creates a clone there, instead of moving the original.

On HalfCheetah, returns vary widely with initial conditions. Admission on a single episode then risks keeping specialists that were merely lucky. There, ENA uses an *Olympic* variant of the Hall of Fame. Every 200,000 environment steps, all gladiators of the current niche replay 3 full episodes on shared seeds, an *Olympiad*. A gladiator is evicted if its mean Olympic return, normalized like Trust, falls below the reference Trust. The others are confirmed, with that mean return. Each niche holds 2 extra seats for gladiators that have not competed yet. Only confirmed gladiators are deployed at test.

Before testing, gladiators are re-injected into the population, round-robin across niches, up to a share of 0.5 of the population. The selection of Section III-I then chooses among them.

H. Environment Detection

The context-detection idea of da Silva et al. [6] is followed. Each niche j owns a model of the environment's dynamics, and a niche is recognized by how well its model predicts the transitions the agent actually experiences. The models are linear-Gaussian:

$$s_{t+1} - s_t = W_j^\top [s_t, a_t, 1] + \varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}\big(0, \mathrm{diag}(\sigma_j^2)\big)\,. \tag{5}$$

They are fitted in closed form, by ridge least squares on accumulated sufficient statistics, so the detector is gradient-free as well. The action is conditioned upon so that the evidence does not depend on which specialist is playing, as long as the model fits the states that specialist visits. Every real transition of every episode is evidence.

1) Detection: The detector monitors the standardized residuals of the current niche's model. Their variance is widened by a tolerance τ^2 ($\tau = 3$), which absorbs the linear model's own error on nonlinear physics. The monitor is the Predictive-Mixture CuSum (PM-CuSum) of Halme, Poor and Koivunen [27], a recent successor of Page's CUSUM [25]. With $q = \mathcal{N}(0, I)$ the pre-change distribution of the residuals z_t,

$$S_t = \max(S_{t-1}, 0) + \log \frac{\hat{p}_t(z_t)}{q(z_t)}, \tag{6}$$

where $\hat{p}_t$ is a Fixed-Share [36] mixture of Gaussian predictors, fitted on sliding windows of $2, 4, \ldots$ recent residuals. An alarm is raised when $S_t > \log\gamma$. Here γ is a target average run length to a false alarm, set to 10^8 steps. For an exact pre-change distribution, Halme et al. show that this threshold keeps the average run length above γ. Here, the pre-change distribution is an online-fitted linear-Gaussian approximation of nonlinear dynamics, so $\log\gamma$ is only a decision threshold. The values of τ and γ were set on preliminary CartPole runs, before the study, so that an unchanged environment raised no alarm. Table XVI reports the false alarms of the study itself.

A departure from the Gaussian example of [27] is made in one respect. Each window also estimates a variance, and that variance is one-sided: it is never below the pre-change variance. The rationale is that a wrong dynamics model can change the spread of the residuals, not only their mean, and should fit worse than the right one, not better.

2) Identification: The recent transitions, up to 500, are matched to the niche j^* with the highest mean per-step log-likelihood. They are recognized as j^* if it explains them within a margin of $m = 2$ nats per step of a model fitted on those same transitions. Otherwise, the physics are new, and a niche is created. Only niches fitted on at least 100 transitions take part.

3) Detect, then verify: An alarm is only a candidate. After 50 post-alarm transitions, the identification test is run against the alarmed niche. If that niche still explains them, the detector treats the alarm as coming from states that its model had not yet covered. The transitions are then added to the model, and the CuSum restarts. Otherwise, a change of physics is declared, and the post-alarm transitions seed the new regime's evidence.

During the test, the dynamics models are never updated. The detector still detects and identifies, but it learns nothing new.

4) Late identity binding: A niche is assigned only when a specialist is considered for the Hall of Fame (step 2 above). The assignment uses the transitions of the episode the specialist just played. The resolved niche's model is refined with them, or a new niche is fitted on them.

5) Chirp detection (HalfCheetah): On HalfCheetah, the transition models misfire. On the detection benchmark of Section V-F, they confirm many more changes, and create many more niches, than there are environments. This happens even when a single fixed gait is played throughout. It happens more often still when the gait changes, as it does in a learning population (another gait, a flip, standing still). The cause was not isolated. Two plausible ones are that a linear model fits the contact-rich dynamics of a running body poorly, and that its evidence is the acting specialist's own transitions.

On HalfCheetah, ENA plays a *chirp* instead: a fixed probing sequence of actions, as in system identification with a known input signal [37]. At every episode end, ENA resets the environment and plays the same 64-step action sequence, whatever any specialist would do. It then compares the observations this produces across niches. Probing an environment to infer its dynamics has also been learned in RL [38]. ENA's chirp is fixed, so its response cannot depend on the agent.

What is played. The excitation is a multiresolution wavelet train. The Mexican hat wavelet is played at dyadic scales, from a segment of half the chirp down to the finest. The aim is to excite both the slow and the fast responses of the body.

What is measured. The response is the observations minus the chirp's starting observation. Each observation channel is decomposed into a Haar wavelet packet tree of depth 3 [39]. Like the discrete wavelet transform [40], the tree is dyadic and non-overlapping, but it splits every band. This gives 8 frequency bands of equal width, and one log-energy is kept per band. Wavelet features of a system's response to a known excitation are a standard signature of a change of physics, in structural damage detection and machine fault diagnosis [41], [42]. The transforms are computed with PyWavelets [43]. The excitation and the features were chosen by the ablation of Section V-F, in the three HalfCheetah training environments. ENA's other HalfCheetah settings were chosen on tuning seeds disjoint from the study's replicates.

How niches are compared. Each niche j keeps the mean μ_j of its n_j chirps' feature vectors. The per-feature variance σ^2 is pooled over niches, on the assumption that it comes mainly from the environment's reset noise. The mean $\bar{f}$ of the last $w = 4$ chirps is tested against the current niche with

$$T_j = \left(\frac{1}{d} \sum_{k=1}^{d} \frac{(\bar{f}_k - \mu_{j,k})^2}{\sigma_k^2 \, (1/w + 1/n_j)} \right)^{1/2}, \tag{7}$$

where d is the number of features. T_j is about 1 under the same physics. Above the margin 3, a change is confirmed, and the most recent chirps identify the new physics. They are assigned to the known niche of lowest T_j within the margin, or else to a new niche (in training only). A niche is judged once it holds 8 chirps. At test, each chirp's Gaussian log-likelihood under each niche is the dynamics evidence L_j of the selection (8).

The chirp costs 64 steps per episode, 6.4 % of a HalfCheetah episode, and these steps count in ENA's budget. The environment's random stream is restored after the chirp, so episode starts are unchanged. CartPole keeps the transition models, which detect its changes (Appendix A-C).

I. Specialist Selection

At test, the weights are frozen and $\mathcal{P} = 0$. The only adaptation left is the choice of which gladiator acts. This choice is made with Bayesian Policy Reuse (BPR) [10]. BPR combines two kinds of evidence in one posterior: the dynamics evidence L_j and the observed utility u. The niches play the role of BPR's types, and every gladiator is a policy. The agent keeps a belief β over niches, and plays the gladiator of highest expected utility, $\sum_j \beta(j)\, \mu_{j\pi}$. Here $u = R/R_{\text{solved}}$ is the return relative to the one at which the environment counts as solved, so solving reads as $u = 1$. $\mu_{j\pi}$ is the mean utility of policy π in niche j. After each episode, the belief is updated:

$$\beta'(j) \propto \beta(j)\, \mathcal{N}\big(u;\, \mu_{j\pi}, 0.2^2\big)\; e^{L_j}. \tag{8}$$

L_j is the log-likelihood of the episode's transitions under niche j's dynamics model (5). It is summed over the episode, with the variance widened by τ^2 as in detection. On HalfCheetah, it is the chirp's log-likelihood (Section III-H5). The return term is bounded by the range of the utilities, while L_j grows with the number of transitions in the episode. When the niches' models differ, the dynamics can therefore dominate the belief over niches. The returns also update the performance model $\mu_{j\pi}$, which decides which gladiator of a niche plays.

Two changes adapt BPR to this setting.

- BPR builds its performance model offline, by playing every policy in every type. The model used here is learned online. Its prior is a utility of 1 for a gladiator in its own niche and 0.5 elsewhere, worth 1 observation. Every observed utility then updates it, weighted by the posterior. On HalfCheetah, the solved return is out of reach. There, a gladiator's own-niche prior is its mean Olympic return relative to R_{solved}, and its prior elsewhere is 0.5 times that.
- BPR assumes one fixed type per trial. To follow a changing environment, a Fixed-Share step [36] mixes back a uniform share after each update: $\beta \leftarrow 0.95\,\beta' + 0.05/|\mathcal{J}|$. A confirmed change of physics also resets the belief to uniform.

At test, the belief starts uniform when the test begins. It is reset to uniform only when the detector confirms a change of physics. As in training, the test environments follow one another unannounced. Three quantities still adapt at test: the belief, the selection's performance model, and the detector's alarms. The weights, Plasticity ($\mathcal{P} = 0$) and the detector's models stay fixed. A confirmed change still resets the input statistics. This matters only for a specialist outside the archive, because a gladiator keeps the statistics it was archived with.

Without a Hall of Fame, there is no gladiator to choose from. The agent then plays a random member of its final population in each test episode. This is how the ablation's variants without a Hall of Fame are tested (Section IV-C1). Section V-D compares BPR with two simpler rules: deploying the highest-Trust gladiator of the niche the detector recognizes, and deploying the highest-Trust gladiator overall.

IV. Experimental Setup

ENA was evaluated in two non-stationary domains built on Gymnasium [44]: the discrete-action `CartPole-v1` [11], and the continuous-control MuJoCo [12] `HalfCheetah-v5`. In both domains, the agent trains on three environments in sequence, without being told when the environment changes. It is then tested, frozen, on five environments, two of which it has never seen.

A. Environments

1) CartPole: The agent balances a pole hinged on a cart by pushing the cart left or right. Each environment changes the gravity g, the pole mass m_p and the pole half-length ℓ (Table II). The environments are named after the planet whose gravity they use. The training environments are also named after their pole (Mars-Light, Jupiter-Heavy, Earth-Standard), and in the text they are abbreviated to Mars, Jupiter and Earth. The training sequence is Mars, Jupiter, Earth. The test sequence interleaves them with two unseen environments: Mars, Neptune, Jupiter, Brown-dwarf, Earth. Neptune is close to Earth. Brown-dwarf is a nominal high-gravity case, with about twice Jupiter's gravity. An episode ends when the pole falls, or after 500 steps. A return of 475 or more counts as a success, which is Gymnasium's registered reward threshold for `CartPole-v1`.

TABLE II
CartPole Environments

Environment	g (m/s^2)	m_p (kg)	ℓ (m)	Seen in training
Mars-Light	3.7	0.05	0.1	yes
Jupiter-Heavy	24.8	0.5	1.5	yes
Earth-Standard	9.8	0.1	0.5	yes
Neptune	11.15	0.1	0.5	no
Brown-dwarf	50	0.1	0.5	no

2) HalfCheetah: The agent controls the six joints of a planar two-legged robot, and is rewarded for running forward. Each environment changes the gravity, the ground friction and the wind (Table III). The friction is applied to every contact surface, since MuJoCo uses the larger friction of two touching bodies. The wind acts through air, which only the windy environments have. The training sequence is Earth (Earth-Standard in the tables) with a soft head wind, then Heavy-Rough (heavier gravity, on rough ground), then Mars-Slippery (Mars's gravity on a slippery floor). The test sequence is Earth-Standard, Ice-Floor, Heavy-Rough, High-Wind and Mars-Slippery. The two unseen environments retain Earth's gravity. Ice-Floor has a much more slippery floor, and neither wind nor air. High-Wind differs from Earth-Standard only by a stronger head wind.

Gymnasium's registered reward threshold for `HalfCheetah-v5` is 4,800, and the figures show it as a reference line. On the non-stationary stream, ENA never reaches it at test, PPO almost never, and SAC in only part of its test episodes (Section V-G). Reliability therefore cannot show forgetting on HalfCheetah. *Retention* and *absolute forgetting* (Section IV-D) replace it as the primary metrics.

TABLE III
HalfCheetah Environments

Environment	g (m/s^2)	Friction	Wind (m/s)	Air (kg/m^3)	Seen
Earth-Standard	9.8	0.4	-5	1.2	yes
Heavy-Rough	14.7	1	0	0	yes
Mars-Slippery	3.7	0.2	0	0	yes
Ice-Floor	9.8	0.05	0	0	no
High-Wind	9.8	0.4	-20	1.2	no

ENA detects HalfCheetah's environments with its chirp (Section III-H5), instead of the transition models.

B. Protocols

1) Skill ablation (CartPole): Each ENA variant (Section IV-C1, Table IV) trains for 500 episodes per environment. It is then tested for 200 episodes per environment. The budget is a number of episodes, not of environment steps. The variants with Plasticity evaluate many more candidates, so they also interact far more with the environment (Section V-A).

A total of 30 independent replicates were run. Replicate k ($k = 1, 2, \dots$) seeds Python's, NumPy's and TensorFlow's random number generators with $1000 + k - 1$ before each agent is built, trained and tested. Every environment is seeded with numbers derived from the same seed. All the agents of a replicate therefore start from the same random state and meet the same initial states: they differ only in what they are. Every study in this paper seeds its replicates the same way.

2) Forgetting in single-policy learners: To measure how much standard learners forget on these streams, single-policy learners train on the same streams as ENA. Comparing their forgetting with ENA's is only fair at equal interaction. Every agent therefore trains on each environment for the same budget of environment steps: 2,500,000 on CartPole and 1,000,000 on HalfCheetah. ENA's budget includes every step it spends evaluating candidates (4), holding Olympiads and playing its chirp (Section III-H5), not only the episodes of its acting specialist.

ENA spends 84.9% of its CartPole training steps evaluating candidates. It therefore plays far fewer episodes within the same budget: about 815 per environment on CartPole, against 6,759 for DQN, 5,791 for PPO and 5,927 for SAC-Discrete. On HalfCheetah, candidate evaluation, Olympiads and chirps take most of its steps. ENA plays about 214 episodes per environment there, against 1,000 for PPO and 1,000 for SAC. Training is compared against environment steps, not episodes, so that every agent spans the same budget. The frozen test needs no such normalization, since every agent

plays the same number of test episodes. No agent observes the physical parameters, or is told when the environment changes. Appendix A lists what each agent receives.

On CartPole, the agent is evaluated, frozen, every 5,000 training steps; ENA is evaluated through its highest-Trust specialist. Each check plays 5 episodes on a separate copy of the current environment. The checks never stop or alter training, and their steps do not count as training. They record the *first solve*: the number of training steps into an environment at which the evaluation mean first reaches the success threshold. A replicate that never solves an environment is counted at the end of its phase. HalfCheetah uses no intermediate checks. First solve is not measured there, since ENA never reaches its threshold and PPO almost never does.

After training, all agents are tested frozen. On CartPole they play 200 episodes per environment, as in the ablation, and on HalfCheetah 50. The CartPole comparison runs 50 replicates. Its 3 comparisons, ENA-Combined against each baseline on test reliability, form one Holm family. This family was planned before the runs. Since the single-policy learners serve only as a reference, the result is read as the overall cost of their forgetting, not as a ranking of the agents. The HalfCheetah comparison runs 20 replicates. Its two comparisons, ENA-Combined against PPO and against SAC on left-behind retention, form one Holm family.

C. Agents

1) ENA variants: The three skills under study are switched on and off as in Table IV, and every other setting is shared. "Plasticity off" means the zero controller (Section III-E3): $\mathcal{P} = 0$, the minimum search. "Plasticity on" means the linear ramp. Environment detection acts only through the Hall of Fame, whose archive it organizes into niches, so it is never tested without it.

At test, the variants with a Hall of Fame choose among their gladiators by Bayesian policy reuse. The variants without one have no gladiator to choose from. They play a random member of their final population in each test episode (Section III-I). Adding the Hall of Fame therefore changes both what is kept and how the test policy is chosen. A random agent gives the floor. The full configuration is listed in Appendix B.

TABLE IV
ENA VARIANTS IN THE SKILL ABLATION

Variant	Plasticity	HoF	Detection
ENA-NoSkill	–	–	–
ENA-Plasticity	✓	–	–
ENA-HoF	–	✓	–
ENA-HoF+EnvDetection	–	✓	✓
ENA-Plasticity+HoF	✓	✓	–
ENA-Combined	✓	✓	✓

2) Single-policy baselines: On CartPole, ENA-Combined was compared with DQN [13], PPO [14] and the discrete-action variant of SAC [45]. On HalfCheetah, it was compared with PPO and SAC [15]. All are custom implementations, with common hyperparameters for these benchmarks (Appendix B). The non-stationary stream made two changes necessary, both found in full-length runs before the reported study:

- *PPO uses a constant learning rate.* With the usual linear annealing to zero over the training budget, PPO reaches its last environment with almost no learning rate left. A learner in a non-stationary stream has no reason to learn less in a later environment than in an earlier one.
- *SAC-Discrete uses a target entropy of* $0.5 \log |\mathcal{A}|$, instead of $0.98 \log |\mathcal{A}|$. With CartPole's two actions, $0.98 \log 2$ is nearly the maximum entropy. The policy cannot reach it, and the temperature keeps growing. Over 148k steps of the first training environment (median of 3 seeds), the temperature rose from 1.0 to 21,048, and the critics' values to 125,066. A ratio of 0.89 diverged in the same way, with a temperature of 4,907 and values of 40,174. At 0.5, the temperature peaked at 3.7 and the values at 121.

During the frozen test, the baselines act deterministically with their final parameters, SAC with the mean of its policy. Section III-I describes what still adapts in ENA at test.

D. Metrics

- *Reliability* (CartPole): the percentage of test episodes whose return reaches the success threshold, over all five test environments. The tables also give it over the three training environments alone, which leaves out the unseen ones.
- *Retention*: the test return as a percentage of the end-of-phase training return. It is computed for each training environment, and averaged over the environments left behind, that is, all training environments except the last. It is a ratio per replicate, so its mean can differ from the ratio of the mean returns. The end-of-phase return is the mean return of the training episodes that end in the last tenth of the phase: of its step budget in the comparisons, and of its episodes in the ablation. Retention is the primary metric on HalfCheetah, and it is reported descriptively on CartPole. End-of-phase returns include every agent's training exploration, so retention can exceed 100%. For an environment the agent never learned, retention measures nothing.
- *Absolute forgetting*: the end-of-phase training return minus the test return, in return units. It is computed for each training environment, and averaged over the environments left behind. Unlike retention, it does not reward an agent for having learned little. Its returns tell a weak policy kept intact from a strong policy partly lost.
- *Mean return* per environment.
- *Steps to first solve* per training environment (CartPole comparison only), at the 5,000-step resolution of the evaluation checks.
- *Compute*: training environment steps, and ENA's candidate-evaluation steps among them, which are properties of the algorithms; and wall-clock training time, which also depends on the implementations and hardware (Appendix C).

Every metric is computed per replicate, and reported as the mean and standard deviation over replicates. Each difference between two agents is reported in three ways: the difference of means with its 95% confidence interval; the effect size d, which is the difference over the root mean square of the two standard deviations; and a Welch's t-test at $\alpha = 0.05$. Steps to first solve are the exception. A few slow replicates dominate their mean, so they are reported as the median and interquartile range, and compared with the two-sided Mann–Whitney U test.

The skill ablation makes 7 planned comparisons: each skill is added on its own, and on top of the others. They are treated as one family, and their p-values are corrected with Holm's method. The number of replicates was fixed before the runs by a power analysis. With 30 replicates per variant, a two-sided test at the family's most stringent level ($0.05/7$) has 80% power for effects of $d \geq 0.94$.

The selection study forms its own Holm family (Section V-D). The Plasticity-controller study forms one family per metric (Section V-C). The HalfCheetah comparison on absolute forgetting, ENA-Combined against PPO and SAC, forms another family, defined after the runs. Retention and absolute forgetting on CartPole, in the ablation and in the comparison, are descriptive and uncorrected, and so is every per-environment comparison.

E. Research Questions and Evidence

Table V maps each research question to the study that answers it. The skill ablation is the primary evidence for RQ1.

TABLE V
RESEARCH QUESTIONS AND THE EVIDENCE FOR EACH

RQ	Evidence	Section
RQ1	skill ablation (reliability and retention); forgetting of single-policy learners	V-A, V-E, V-G
RQ1a	ablation: Hall of Fame with and without detection; selection ablation	V-A, V-D
RQ1b	ablation: candidate steps with and without the archive	V-A
RQ1c	recognition per phase (both domains); detection benchmark (HalfCheetah)	A-C, V-F
RQ2	ablation; compute study; controller study	V-A, V-B, V-C

V. RESULTS

This section first evaluates the individual contributions of ENA's constituent skills and its specialist selection mechanism through ablation studies on CartPole. Performance is subsequently analyzed on previously encountered and unseen environments alongside single-policy baselines, accompanied by the detection benchmarks on HalfCheetah. Finally, the interaction costs of failure-driven search and alternative controller formulations are assessed. Table V maps each subsection to the research question it informs. Reporting conventions and statistical tests are detailed in Section IV-D; unless stated otherwise, reported values are means over independent replicates with standard deviations in parentheses, and significance is assessed via Welch's two-sided t-test.

A. Skill Ablation

Table VI and Fig. 2 report the frozen test over 30 replicates. With all three skills, **ENA-Combined** achieved a test reliability of 83.2% (16.1). Without any skill, ENA-NoSkill achieved 38.5% (20.0). The difference is +44.7 points (95% CI 35.3 to 54.1; $d = 2.46$; $p < 0.001$, uncorrected). ENA-Combined is more reliable than every other variant. Its smallest margin is over ENA-Plasticity: +19.9 points (11.9 to 27.9; $p < 0.001$, uncorrected).

Table VII gives the 7 planned comparisons, which add one skill at a time. They form one Holm family, and each difference is in points of test reliability. In all three pairs of variants that differ only by Plasticity, the variant with Plasticity is more reliable. In both pairs that differ only by environment detection, the variant with detection is more reliable. ENA-HoF is less reliable than ENA-NoSkill (-13.8 points, Holm $p = 0.027$), and ENA-Plasticity+HoF does not differ significantly from ENA-Plasticity (-4.4, Holm $p = 0.408$). As Section IV-C1 notes, each Hall of Fame comparison also changes how the test policy is chosen.

Acquisition and retention. Reliability reflects both performance acquisition within an environment and retention across subsequent environment switches. Table VIII separates these aspects on the environments left behind, Mars and Jupiter. It reports each variant's end-of-phase and test returns, its retention, and its absolute forgetting. Without Plasticity, ENA achieves minimal learning on Jupiter. ENA-NoSkill ends its Jupiter phase at a return of 71, and scores 144.8 there at test. Its retention above 100% indicates only that no acquired capability was degraded. The three variants with Plasticity acquire policies in both environments, but retention differs across architectures. ENA-Plasticity retains 58.1% of its Mars return and 75.4% of its Jupiter return. ENA-Plasticity+HoF retains Mars (100.5%), but only 58.8% of Jupiter. ENA-Combined maintains performance in both, with 101.5% and 97.4%. These retention values are descriptive.

The archive organized by detection can also be compared with no archive at all. With Plasticity, ENA-Combined is more reliable than ENA-Plasticity, as reported above. Without Plasticity, the difference is not resolved: ENA-HoF+EnvDetection is +9.7 points above ENA-NoSkill (-3.6 to 23.0; $p = 0.148$, uncorrected).

Unseen environments. On Neptune, ENA-Combined's mean return is 490.4. Its detector recognizes Neptune as Earth's niche in 30 of the 30 replicates. On Brown-dwarf, its mean return is 367.2 (153.9), and the detector leaves the environment unrecognized in 30 of them. There, it also confirms 16.4 changes per replicate on average (18.8). At most one of them follows the real switch, and each one resets the selection's belief to uniform. Table XVI (Appendix A-C) gives the recognition per phase: confirmed changes, unrecognized episodes and niches per environment.

Interaction and cost of the archive. The ablation fixes episodes, not steps, so the variants with Plasticity interact far more with the environment. ENA-NoSkill, ENA-

HoF and ENA-HoF+EnvDetection take 266,057, 267,760 and 343,189 training steps per replicate. ENA-Plasticity, ENA-Plasticity+HoF and ENA-Combined take 4,532,688, 4,893,572 and 4,920,196.

Adding the archive and detection changes the candidate-evaluation steps as follows (exploratory, uncorrected). Without Plasticity, ENA-NoSkill spends 28,145 candidate steps per replicate. Adding the Hall of Fame changes this by +227 (-57 to 511; $p = 0.115$), and adding detection on top changes it by -111 (-340 to 119; $p = 0.338$). With Plasticity, ENA-Plasticity spends 3,841,713 candidate steps. Adding the Hall of Fame changes this by +373,159 ($-27{,}667$ to 773,986; $p = 0.067$), and adding detection on top changes it by +12,049 ($-361{,}749$ to 385,847; $p = 0.949$). Wall-clock training takes 6.0, 6.2 and 7.5 minutes per replicate for ENA-NoSkill, ENA-HoF and ENA-HoF+EnvDetection.

Statistical power. With 30 replicates, the family detects $d \geq 0.94$ with 80% power (Section IV-D). The five comparisons that add Plasticity or detection have d between 0.89 and 1.47. Each of their intervals excludes zero, as does ENA-Combined's margin over each variant. The Hall of Fame's effect on ENA-NoSkill is significant, but smaller than the design detects ($d = -0.66$). Measuring it with 80% power would take 60 replicates. Its effect on ENA-Plasticity ($d = -0.22$) would take 539.

TABLE VI
SKILL ABLATION: FROZEN TEST (30 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)	Train rel. (%)
Random	9.0	21.8	33.7	16.3	22.2	0.0 ± 0.0	0.0
ENA-NoSkill	311.8	321.7	144.8	170.1	321.4	38.5 ± 20.0	40.1
ENA-Plasticity	279.8	416.3	350.7	313.1	419.4	63.3 ± 14.8	63.6
ENA-HoF	326.6	252.4	128.3	108.3	299.9	24.7 ± 21.9	30.2
ENA-HoF+EnvDetection	404.5	393.7	272.5	208.5	414.8	48.3 ± 30.3	53.7
ENA-Plasticity+HoF	474.8	352.1	277.9	281.5	450.7	58.9 ± 24.6	69.9
ENA-Combined	475.5	490.4	464.9	367.2	490.5	83.2 ± 16.1	87.5

TABLE VII
SKILL ABLATION: PLANNED COMPARISONS (30 REPLICATES)

Skill added	To	Difference	95% CI	d	Holm
Plasticity	ENA-NoSkill	+24.8	15.7 to 33.9	1.41	$p < 0.001$
Hall of Fame	ENA-NoSkill	-13.8	-24.6 to -2.9	-0.66	$p = 0.027$
Detection	ENA-HoF	+23.5	9.8 to 37.2	0.89	$p = 0.003$
Hall of Fame	ENA-Plasticity	-4.4	-14.9 to 6.2	-0.22	$p = 0.408$
Plasticity	ENA-HoF	+34.2	22.1 to 46.2	1.47	$p < 0.001$
Detection	ENA-Plasticity+HoF	+24.3	13.5 to 35.1	1.17	$p < 0.001$
Plasticity	ENA-HoF+EnvDetection	+35.0	22.4 to 47.6	1.44	$p < 0.001$

B. Compute Cost of Plasticity

Variants with Plasticity train in 34.9–38.9 minutes per replicate, far longer than those without it, and they spend far more environment steps evaluating candidates (Section V-A).

ENA-Combined was evaluated once with the full controller ($\mathcal{P} = 1$, Section III-E3) on the first replicate of the ablation. This execution was compared with the same replicate under the linear ramp (ENA-Combined) and under the zero controller (ENA-HoF+EnvDetection). During training, the three runs

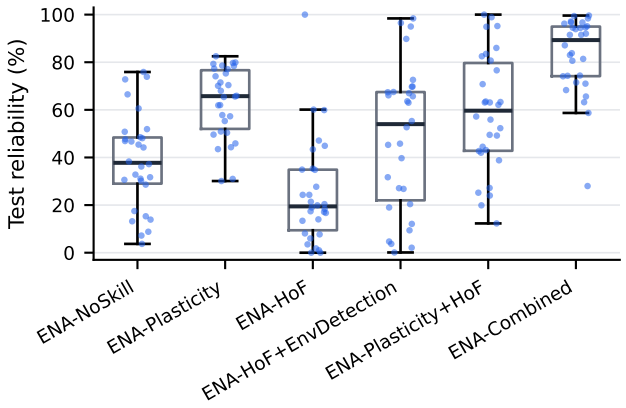


Fig. 2. Test reliability of each ENA variant; each dot is one of the 30 replicates.

spent 57,890,361, 4,617,335 and 27,816 environment steps evaluating candidates. Full search costs 13 times the linear ramp, which costs 166 times the minimum. At full effort, every generation evaluates every other specialist for up to $s_{\max}$ steps, so its cost is fixed by construction. The linear ramp's cost varies across replicates, with a standard deviation of 596,722 steps over the ablation's 30. The ratios above are therefore those of this replicate.

In this replicate, test reliability was 90.3% at full effort, 74.1% with the ramp, and 55.3% at minimum search. Most of the gap between full effort and the ramp comes from Brown-dwarf, the extrapolation environment, with mean returns of 410.0 and 61.5. A single replicate supports no test. Among the ramp's 30 ablation replicates, 15 reached at least the reliability of full search, and this replicate is one of the ramp's weaker runs. Whether the ramp matches the reliability of full search therefore remains open.

C. Shape of the Plasticity Controller

The linear ramp controller (1) was compared with the two alternatives from Section III-E3: the quadratic and the fuzzy controller. Each alternative replaced only ENA-Combined's Plasticity controller, and ran for 30 replicates with the ablation's seeds. Every agent of a replicate starts from the same random state. The ablation's ENA-Combined is therefore the linear-ramp run of the same replicates, and serves as the reference without being re-run.

The decision rule was fixed before the runs. An alternative replaces the ramp if its test reliability is significantly higher, or if it is not significantly lower and uses significantly fewer candidate-evaluation steps. For each metric, the two comparisons with the ramp form one Holm family. At 30 replicates, each family has 80% power for $d \geq 0.81$. Table IX gives the comparisons.

The ramp achieved a test reliability of 83.2% (16.1), with 4,226,921 candidate steps per replicate. The fuzzy controller achieved 83.9% (11.4), which is not distinguishable from the

TABLE VIII
SKILL ABLATION: RETENTION OF THE ENVIRONMENTS LEFT BEHIND (30 REPLICATES)

Agent	Mars end	Mars test	Mars ret. (%)	Jupiter end	Jupiter test	Jupiter ret. (%)	Retention (%)	Forgetting
ENA-NoSkill	146	312	499.5	71	145	250.8	375.1	−120
ENA-Plasticity	483	280	58.1	468	351	75.4	66.7	160
ENA-HoF	274	327	178.3	60	128	272.9	225.6	−61
ENA-HoF+EnvDetection	272	404	249.0	159	273	236.3	242.6	−123
ENA-Plasticity+HoF	473	475	100.5	468	278	58.8	79.6	94
ENA-Combined	469	476	101.5	478	465	97.4	99.5	3

ramp's, and it spent significantly more candidate steps. The quadratic controller achieved 89.3% (7.7) with fewer candidate steps, but neither difference is significant. Its effect on reliability ($d = 0.48$) is below the detectable one. Resolving it with 80% power would take about 84 replicates per controller.

By this criterion, the controller retained is the linear ramp. As an exploratory observation outside the planned comparisons, the quadratic controller's reliability varied less across replicates, with a standard deviation of 7.7 against 16.1.

TABLE IX
PLASTICITY CONTROLLERS AGAINST THE LINEAR RAMP (30 REPLICATES)

Controller	Metric	Difference	95% CI	d	Holm
Quadratic	Reliability (points)	+6.0	−0.5 to 12.6	0.48	$p = 0.141$
Quadratic	Candidate steps	−294,127	−633,393 to 45,140	−0.45	$p = 0.088$
Fuzzy	Reliability (points)	+0.7	−6.5 to 7.9	0.05	$p = 0.854$
Fuzzy	Candidate steps	+481,852	187,089 to 776,616	0.85	$p = 0.004$

D. Selection Ablation

ENA-Combined deploys its archived specialists at test with Bayesian policy reuse (Section III-I). Two simpler drivers replaced only that choice. *Niche match* plays the highest-Trust gladiator of the niche the detector currently recognizes. When that niche has no gladiator, it plays the highest-Trust gladiator overall. It ignores the test returns. *Best Trust* always plays the highest-Trust gladiator overall, ignoring both the detector and the returns.

Training selection is the same for every driver. Each alternative therefore ran for 30 replicates with the ablation's seeds, and the ablation's ENA-Combined is the Bayesian reference without being re-run. In 60 of the 60 alternative runs, the training log is byte-identical to the reference's. Two comparisons, fixed before the runs, form one Holm family: Bayesian policy reuse against niche match, and niche match against best Trust. At 30 replicates, the family has 80% power for $d \geq 0.81$.

Table X reports the frozen test. Test reliability is 83.2% (16.1) with Bayesian policy reuse, 59.2% (25.9) with niche match, and 53.8% (32.3) with best Trust. Each difference is given with its 95% interval and Holm-corrected p-value:

- *Niche match against Bayesian policy reuse*: −24.0 points (−35.2 to −12.8; $d = -1.11$; $p < 0.001$). Niche match is lower in 29 of the 30 replicates.
- *Best Trust against niche match*: −5.4 points (−20.6 to 9.7; $d = -0.19$; $p = 0.476$). Best Trust is lower in 15 replicates and higher in 15.

Over a test, Bayesian policy reuse plays 7.2 distinct individuals on average, niche match 3.2, and best Trust 1.0. On Jupiter, the training environment left behind, niche match's mean return is 407.4 and best Trust's is 206.8 (−200.6; −283.2 to −118.0; $p < 0.001$, uncorrected). On every other environment, the interval of their difference includes zero.

Archive without Bayesian policy reuse. ENA-Plasticity maintains no archive, and plays a random member of its population at test (Section IV-C1). It was evaluated with the same seeds, allowing a comparison with the two simpler drivers. Its test reliability is 63.3%. Against it, niche match differs by −4.1 points (−15.0 to 6.9; $p = 0.459$, uncorrected), and best Trust by −9.5 points (−22.6 to 3.6; $p = 0.151$, uncorrected). With either driver, the archive does not provide greater reliability than operating without an archive. These comparisons are exploratory. The agents also differ during training, so they do not isolate the archive.

TABLE X
SELECTION ABLATION: FROZEN TEST (30 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)	Train rel. (%)
ENA-Combined	475.5	490.4	464.9	367.2	490.5	83.2 ± 16.1	87.5
ENA-Combined-NicheMatch	396.5	384.8	407.4	235.6	397.1	59.2 ± 25.9	67.7
ENA-Combined-BestTrust	403.2	388.3	206.8	239.7	390.4	53.8 ± 32.3	55.9

E. Forgetting in Single-Policy Learners: CartPole

Tables XI and XII and Fig. 3 report 50 replicates, under the equal budget of 2,500,000 environment steps per environment. Appendix D shows the episode-by-episode test curves across all five environments (Fig. 5).

1) Forgetting: The single-policy learners end training on Earth. On Jupiter, the environment they trained on just before it, their training return at the end of the phase is 498 (PPO), 493 (SAC-Discrete) and 404 (DQN). At test, they score 21.6, 52.1 and 75.2 there. They retain 4.3%, 10.5% and 19.9% of their return, an absolute forgetting of 477, 441 and 329. ENA-Combined ends Jupiter at 468, and scores 461.3 there at test (98.6%; forgetting 7). At test, it recognizes Jupiter's niche in 50 of the 50 replicates, and mostly plays Jupiter's gladiators in 45. On Mars, the first training environment, every agent preserves its end-of-phase return: PPO 101.1%, SAC-Discrete 100.1%, DQN 98.7% and ENA-Combined 103.2%. Over the

three training environments alone, test reliability is 88.6% for ENA-Combined, against 66.3% for SAC-Discrete, 65.5% for PPO and 45.9% for DQN (Table XI). These values are descriptive.

2) Reliability: ENA-Combined's test reliability is 83.9% (11.7), against 74.5% for SAC-Discrete, 73.8% for PPO and 50.6% for DQN (Fig. 3). Each of the three differences is significant after Holm correction over the 3 comparisons:

- *SAC-Discrete:* +9.4 points (95% CI 5.0 to 13.7; $d = 0.85$; Holm $p < 0.001$).
- *PPO:* +10.1 points (95% CI 5.8 to 14.3; $d = 0.94$; Holm $p < 0.001$).
- *DQN:* +33.3 points (95% CI 25.1 to 41.4; $d = 1.63$; Holm $p < 0.001$).

On the environments other than Jupiter, SAC-Discrete's test return is slightly higher than ENA-Combined's. On Mars, it is 495.7 against 480.2 ($p = 0.026$). On Earth, it is 500.0 against 498.1 ($p = 0.049$), and on the unseen Neptune 500.0 against 492.9 ($p < 0.001$). On Brown-dwarf, the other unseen environment, ENA-Combined scores 381.6, against 379.5 for PPO and 393.5 for SAC-Discrete ($p = 0.944$ and $p = 0.695$). These per-environment comparisons are uncorrected and descriptive.

3) Adaptation: Table XII gives the median steps to first solve, with their interquartile range. ENA-Combined never solved Jupiter in 1 of the 50 replicates. PPO's median first solve takes 10,018, 47,743 and 5,000 steps on the three training environments. ENA-Combined's takes 44,841, 106,116 and 39,288, later on all three ($p < 0.001$, $p = 0.002$ and $p < 0.001$). Against the other two baselines:

- *Mars:* ENA-Combined solves it sooner than both DQN (257,626 steps; $p < 0.001$) and SAC-Discrete (127,650; $p = 0.025$).
- *Jupiter:* sooner than SAC-Discrete (162,720; $p = 0.002$), and not distinguishably from DQN (122,843; $p = 0.095$).
- *Earth:* later than both (5,476 and 5,000; $p < 0.001$ and $p < 0.001$).

4) Compute: With the same number of environment steps, ENA-Combined trained in 65.3 minutes of wall-clock time per replicate. DQN took 97.1 minutes, SAC-Discrete 119.0 and PPO 140.3, on the same CPU and with implementations in the same framework. ENA computes no gradients: its cost is forward passes and environment steps.

TABLE XI
CARTPOLE COMPARISON: FROZEN TEST (50 REPLICATES)

Agent	Mars	Neptune	Jupiter	Brown	Earth	Rel. (%)	Train rel. (%)
DQN	401.6	443.7	75.2	238.2	442.0	50.6 ± 26.4	45.9
PPO	488.0	497.9	21.6	379.5	498.0	73.8 ± 9.6	65.5
SAC-Discrete	495.7	500.0	52.1	393.5	500.0	74.5 ± 10.2	66.3
ENA-Combined	480.2	492.9	461.3	381.6	498.1	83.9 ± 11.7	88.6

F. Environment Detection on HalfCheetah

The detector's inputs are the agent's own, so detection can be evaluated without training. The detection benchmark plays scripted episodes in Earth, Heavy-Rough and Mars-Slippery,

TABLE XII
CARTPOLE COMPARISON: TRAINING STEPS TO FIRST SOLVE, MEDIAN (IQR) (50 REPLICATES)

Agent	Mars	Jupiter	Earth
DQN	257,626 (201,536–295,276)	122,843 (100,210–173,903)	5,476 (5,000–30,069)
PPO	10,018 (6,282–10,046)	47,743 (35,289–75,043)	5,000 (5,000–5,000)
SAC-Discrete	127,650 (25,366–184,064)	162,720 (146,504–175,217)	5,000 (5,000–5,000)
ENA-Combined	44,841 (20,205–69,308)	106,116 (36,117–204,861)	39,288 (26,760–55,594)

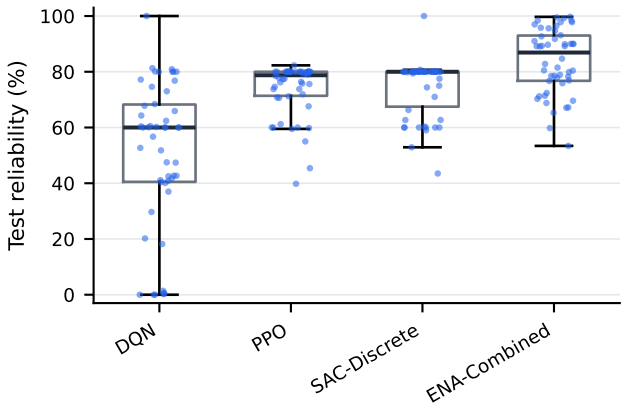


Fig. 3. Test reliability on CartPole across 50 replicates, under equal budgets of environment steps; each dot is one replicate.

40 in each. Every 4 episodes, it changes the acting policy among near-still, uniformly random and sinusoidal gaits. The same family of gaits is used in every environment, so that only the physics differ. The benchmark feeds the detector exactly the calls ENA makes, with a niche assignment after every episode. A run meets the target when three conditions hold. It confirms both switches within 5 episodes. It raises no false alarm inside an environment. It gives each environment its own niche, covering at least 90 % of that environment's episodes.

Each detector was evaluated on 30 independent benchmark runs. Each run has its own random seed, which draws different policies and resets, so each run is identified by its seed. Intervals are exact (Clopper–Pearson) at 95 %. Comparisons are exact McNemar tests on seed-paired outcomes, Holm-adjusted as one family. As a check of the benchmark's criteria, CartPole's transition-model detector meets the target on 26 of 30 runs of the corresponding CartPole benchmark.

On HalfCheetah, the transition-model detector is calibrated for the domain: margin $m = 3$, at least 1,000 transitions to fit a niche, and 1,000 to verify an alarm. It meets the target on 0 of 30 runs. On average, it confirms 26.3 changes, 24.5 of them inside an environment, and creates 21.8 niches. Only 21.9 % of the worst environment's episodes fall in its niche. With one fixed gait in every environment, the same detector still confirms 16.2 changes and creates 6.5 niches on average.

It also merges two environments into one niche on 14 of the 30 seeds.

Table XIII ablates the chirp's excitation and response features over 30 benchmark seeds. Each cell counts the seeds on which that pair meets the target, and ENA's chirp is the best of them. The Mexican-hat wavelet train is the best excitation. It meets the target on at least 29 of the 30 seeds with every response feature, and on every seed with 3 of them. No linear frequency sweep, sine or impulse meets the target on any seed. The best configuration without a wavelet train, an exponential sweep with SWT Haar features, meets it on 4 seeds. The Haar and db4 [46] trains depend strongly on the features. ENA uses the Mexican-hat train with Haar wavelet packets (in bold), which meets the target on every seed.

On the confirmation runs, this chirp meets the target on 29 of 30 seeds (95 % interval 82.8–99.9 %). It confirms a false change on 1 of them, and confirms every switch at the end of the first episode after it. The share of the worst environment's episodes that fall in its own niche averages 99.2 % over the seeds, and is never below 75 %. The chirp meets the target more often than the transition-model detector ($p < 0.001$). On the 30 ablation seeds it was chosen on, it also meets the target more often than the best configuration without a wavelet train ($p < 0.001$).

In the HalfCheetah study itself (Section V-G), the chirp gives each training environment its own niche in 20 of the 20 replicates (Table XVII, Appendix A-C). A training run, with two switches, confirms 3.9 changes on average. A test, with four switches, confirms 9.1. At test, the training environments are recognized as their own niches in most replicates: Earth-Standard in 20, Heavy-Rough in 17 and Mars-Slippery in 18. High-Wind is recognized as Earth-Standard's niche in 20 replicates. Ice-Floor is left unrecognized in 52.2 % of its episodes.

TABLE XIII
CHIRP DETECTOR ABLATION (30 SEEDS)

Excitation	Raw	DWT Haar	DWT db4	SWT Haar	Packets Haar
Linear sweep	0/30	0/30	0/30	0/30	0/30
Exponential sweep	0/30	0/30	0/30	4/30	1/30
Sine	0/30	0/30	0/30	0/30	0/30
Impulse	0/30	0/30	0/30	0/30	0/30
Wavelet train (Haar)	26/30	12/30	28/30	2/30	14/30
Wavelet train (db4)	0/30	12/30	17/30	13/30	26/30
Wavelet train (Mexican hat)	30/30	29/30	30/30	29/30	**30/30**

G. Forgetting in Single-Policy Learners: HalfCheetah

Tables XIV and XV and Fig. 4 report 20 replicates. On the non-stationary stream, ENA-Combined reaches the 4,800 threshold in 0.0% of its test episodes, PPO in 0.9%, and SAC in 47.1%. As explained in Section IV, retention and absolute forgetting are the primary metrics here. Appendix D shows the episode-by-episode test curves and the training curves (Figs. 6 and 7).

1) Forgetting: The environments left behind are Earth-Standard, trained first, and Heavy-Rough, trained second. PPO ends Heavy-Rough at 4,793 and scores 1,302 there at test, retaining 28.3%. SAC ends it at 7,544 and scores 1,301, retaining 18.0%. Averaged over both environments left behind, PPO loses 2,913 (965) of its end-of-phase return, and retains 39.2% (17.9). SAC loses 5,263 (1,234), and retains 34.4% (13.1).

ENA-Combined's absolute forgetting is -239 (229), and its retention 115.6% (16.6). Its test returns, 2,000 on Earth-Standard and 2,417 on Heavy-Rough, exceeded its end-of-phase training returns of 1,805 and 2,133. On Heavy-Rough at test, it mostly plays Heavy-Rough's gladiators, in 20 of the 20 replicates.

The planned comparisons on retention, Holm-corrected over the two baselines, are:

- *PPO:* +76.3 percentage points (95% CI 65.3 to 87.4; $d = 4.42$; Holm $p < 0.001$).
- *SAC:* +81.2 percentage points (95% CI 71.6 to 90.8; $d = 5.44$; Holm $p < 0.001$).

Absolute forgetting forms a Holm family defined after the runs. On it, ENA-Combined's difference is $-3{,}152$ against PPO ($-3{,}613$ to $-2{,}692$; Holm $p < 0.001$), and $-5{,}502$ against SAC ($-6{,}087$ to $-4{,}917$; Holm $p < 0.001$).

ENA-Combined's end-of-phase return also includes episodes played by specialists outside its archive, such as the ϵ-greedy selection's picks. Its test plays only gladiators. In 18 of the 20 replicates, gladiators played at the end of both phases left behind. Measured against the gladiators' end-of-phase episodes alone, ENA-Combined's retention in these replicates is 103.2% (10.5), and its absolute forgetting -26. The single-policy learners' end-of-phase returns also include their exploration, since they act deterministically only at test. These values are descriptive.

2) Absolute return: SAC's test returns exceeded ENA-Combined's on every environment but Heavy-Rough. On Mars-Slippery, SAC scores 10,146 against 1,773 ($p < 0.001$). On Ice-Floor, it scores 6,774 against 1,915 ($p < 0.001$), and on High-Wind 2,642 against 996 ($p < 0.001$). On Earth-Standard, SAC still scores 4,353 against 2,000 ($p < 0.001$), even after losing half of its end-of-phase return there. SAC's end-of-phase returns on the environments left behind, 8,635 and 7,544, are also several times ENA-Combined's. On Heavy-Rough at test, ENA-Combined's return is higher than SAC's (+1,116; $p < 0.001$) and PPO's (+1,115; $p < 0.001$). These per-environment comparisons are uncorrected and descriptive (Table XV).

3) Compute: ENA-Combined trained in 32.0 minutes per replicate (1.7). PPO took 115.3 minutes and SAC 281.1, on the same instance type (Appendix C). ENA's budget was 1,000,000 training steps per environment. Over the three environments, 2,376,493 of its training steps went to candidate evaluation, Olympiads and chirps, and its acting specialist played the rest (Section IV-B2).

TABLE XIV
HALFCHEETAH COMPARISON: RETENTION OF THE ENVIRONMENTS LEFT BEHIND (20 REPLICATES)

Agent	Earth end	Earth test	Earth ret. (%)	Heavy end	Heavy test	Heavy ret. (%)	Retention (%)	Forgetting	Time (min)
PPO	4,543	2,208	50.1	4,793	1,302	28.3	39.2	2,913	115.3
SAC	8,635	4,353	50.8	7,544	1,301	18.0	34.4	5,263	281.1
ENA-Combined	1,805	2,000	116.3	2,133	2,417	114.9	115.6	−239	32.0

TABLE XV
HALFCHEETAH COMPARISON: FROZEN TEST (20 REPLICATES)

Agent	Earth	Ice	Heavy	High	Mars
PPO	2,208 ± 1,013	2,203 ± 1,000	1,302 ± 1,087	929 ± 908	2,506 ± 1,187
SAC	4,353 ± 981	6,774 ± 2,214	1,301 ± 1,218	2,642 ± 896	10,146 ± 975
ENA-Combined	2,000 ± 492	1,915 ± 503	2,417 ± 539	996 ± 511	1,773 ± 375

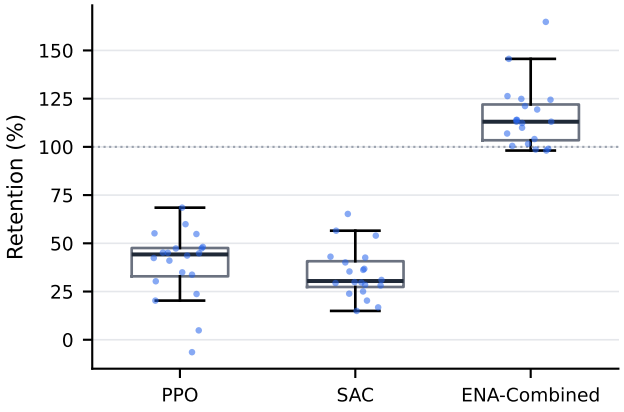


Fig. 4. Left-behind retention on HalfCheetah (20 replicates); each dot is one replicate. The dotted line marks 100%.

VI. DISCUSSION

A. Answers to the Research Questions

RQ1: do the skills mitigate forgetting? Supported on CartPole, for the tested sequences. In the ablation, the variants that learn Mars and Jupiter are those with Plasticity. Among them, only ENA-Combined maintains performance in both environments at test, and it is the most reliable variant (Section V-A). Adding the archive also changes how the test policy is chosen, so the ablation does not isolate the archive's contribution from the selection's. Without Plasticity, ENA fails to acquire competent policies on Jupiter. The reliability gains observed in the ablation therefore reflect both policy acquisition and subsequent retention. The single-policy learners degrade markedly on Jupiter, the CartPole environment trained just before the last, though they retain performance on Mars. On HalfCheetah, they keep 34.4–39.2% on average of what they reached on the two environments left behind, whereas ENA retains full performance across all previously trained dynamics (Sections V-E and V-G). All of these findings hold for the specific implementations, task sequences, and budgets evaluated; alternative settings were not tested.

RQ1a: is an archive enough by itself? No. Variants with a Hall of Fame but no detection are not more reliable than the same variants without it, and ENA-HoF is less reliable than ENA-NoSkill. Variants whose archive is organized by detected physics are more reliable than those with an archive but no detection, in both cases. With Plasticity, they are also more reliable than the variant without any archive, and only this combination maintains performance across all learned environments (Section V-A). In this ablation, merely archiving proven specialists proved insufficient: archiving increased reliability only when indexed by the physical regime under which each specialist was validated. The recall rule also matters. Bayesian policy reuse is more reliable than deploying the recognized niche's highest-Trust gladiator. Niche match, in turn, is not significantly more reliable than ignoring the niche (Section V-D). With either simpler rule, the archive is not more reliable than no archive at all.

RQ1b: does the memory cost extra interaction? Partly answered. Without Plasticity, archiving and detecting add no measurable candidate evaluation of their own. Their only measured cost is wall-clock time (Section V-A). With Plasticity, adding the Hall of Fame comes with more candidate steps. The increase is not significant, but the data cannot exclude it, up to 773,986 steps per replicate. The archive takes no extra candidate evaluation to fill, but it may change how much the failure-driven search spends.

RQ1c: can the memory be built online, without labels? Yes in both domains, with different drivers. On CartPole, the transition-model detector separates the training environments, with the fragmentation and false alarms of Table XVI. On HalfCheetah, the same kind of detector fails (Section III-H5). The chirp, whose response does not depend on the agent, meets the detection target on the benchmark. It also gives each training environment its own niche in every replicate of the HalfCheetah study (Section V-F). Neither driver uses a gradient.

RQ2: does failure-driven search pay? Partly supported. The comparisons that add Plasticity show the largest differences in the ablation, and only the variants with Plasticity learn the harder environments (Section V-A). In the ablation, however, the budget is a number of episodes, so more search also means far more interaction with the environment. Searching at full effort in every episode costs an order of magnitude more candidate evaluation (Section V-B), a cost set by construction. Whether the ramp reaches what full effort does remains open. In the single replicate run at full effort, full effort was the more reliable. Whether scaling search effort dynamically with failure outperforms distributing identical effort uniformly

across iterations remains an open empirical question. The quadratic controller was more reliable than the ramp and used fewer candidate steps, but neither difference is significant (Section V-C). These findings therefore support the general principle of scaling search with failure, rather than the specific linear ramp evaluated here.

B. How Detection May Help the Archive

The outcome of RQ1a is interpreted as a limitation of recall rather than of storage capacity. The selection ablation is consistent with this reading, but neither ablation tests it directly. Without niches, the archive is one pool of gladiators proven in different physics. During training, their protected copies hold population slots, whatever the current physics. At test time, the selection mechanism lacks dynamical evidence to distinguish specialists evolved for Jupiter from those evolved for Earth. Adding the archive also replaces a random population member by Bayesian policy reuse (Section IV-C1). With detection, every gladiator carries the physics it was proven in. The dynamics likelihoods of the selection (8) point it to the matching niche, and the gladiators of one niche can no longer be evicted by another niche's.

Selection rules that use the same niches also differ in reliability. Bayesian policy reuse differs from niche match in two ways. It weighs each episode's dynamics likelihoods into a belief, instead of following the detector's single decision. It also learns from the test returns. The selection ablation does not separate the two. One plausible mechanism, not directly isolated in the experiments, involves the feedback provided by episode returns. They let Bayesian policy reuse abandon a gladiator that fails in its own niche, which niche match cannot do, and across a test phase, niche match evaluates substantially fewer distinct individuals (Section V-D). A gladiator archived for solving a niche could still fail there at test, and without return-based evidence, an ineffective specialist would continue to be deployed.

C. Retention, Adaptation and Absolute Return

ENA reliably preserves acquired behaviors, but exhibits lower sample efficiency and asymptotic return than single-policy learners in other respects, by a margin that varies with the domain. Both gaps match known limits of evolutionary search. Evolutionary algorithms need many samples, and struggle on problems that require optimizing many parameters [19], [20]. The cost of exploring in parameter space grows with the number of parameters [47], and random mutations tend to disrupt large networks [48]. This is why ENA's specialists are small. On CartPole, ENA reaches criterion performance in a new environment later than PPO, as the majority of environment steps are spent evaluating candidate perturbations prior to identifying a successful policy (Section IV-B2). SAC-Discrete scores slightly higher on the environments it does not forget, whereas ENA preserves the earlier environment on which single-policy baselines catastrophically degrade.

On HalfCheetah, the gap is larger. ENA preserves performance from earlier phases, but achieves lower asymptotic returns than SAC within the allocated step budget. The robot has six joints to control. PPO and SAC control them with hidden layers of 256×256 ReLU units, while ENA's specialists have 32×16 tanh units. Evolving networks as large as the baselines' would mean searching far more parameters, at the cost described above. Even with small networks, most of ENA's steps go to evaluating candidates rather than to its acting specialist's episodes (Section IV-B2). What SAC loses on the environments left behind exceeds the return ENA reached there at the end of each phase (Section V-G). Most of ENA's retention above 100% comes from its end-of-phase reference, which includes episodes of specialists outside the archive. Measured against its gladiators' episodes alone, its retention is 103.2% (Section V-G).

A specialist archive is hypothesized to provide the greatest advantage when environments recur and relearning is computationally expensive, though this setting was not explicitly evaluated. When an environment is unlikely to return, or a gradient learner can relearn it quickly, a single strong policy that is partly forgotten may be worth more than a weaker one kept intact.

D. Scope of the Comparison

The single-policy learners measure how much standard learners forget on these streams. The claim of this paper rests on the skill ablation. The comparison has three limits. First, at test, every agent's weights are frozen, but ENA still routes among its archived specialists with an updating belief. The single-policy learners deploy one policy. The comparison thus contrasts context-indexed specialist preservation with single-policy learning, not two equally equipped systems.

Second, ENA evaluates its candidates in fresh copies of the current environment, which the experiment builds from the environment's own parameters (Appendix A). Every candidate step counts in ENA's budget. Still, the single-policy learners play only in the stream itself, and never use such a copy.

Third, the continual-RL methods closest to ENA are not baselines here, for reasons that differ between them. OWL [5] and TeLAPA [24] are told where tasks begin and end. OWL trains the head of the task it is given, and infers the task only at test. TeLAPA builds a task's archive, and retrieves a starting point from its archives, at each task boundary it is given. Both build their memory during the stream, but from labeled tasks. On the non-stationary streams evaluated here, these methods would require either explicit task boundaries—which are unavailable to any agent in this study—or an environment detector to infer them. Providing such an inferential mechanism would effectively endow those baselines with ENA's environment-detection capability. ITE [23] builds its archive offline, before deployment. It adapts once to each damage, starting when performance drops below a user-set threshold. It neither keeps a memory across a sequence of conditions, nor recognizes a condition that returns.

RL-CD [6], MBCD [7] and SWOKS [8] do detect contexts online without labels, and keep a policy per context, so they address the same problem. A comparison with them,

however, would be confounded. RL-CD's models are tabular and need a discretized state space, while MBCD and SWOKS learn by gradient descent. A difference could therefore come from the learning algorithm rather than from the memory. Direct comparisons with such algorithms are deferred to future investigations of hybrid architectures combining ENA's modular skills with gradient-based optimization (Section VII). One promising formulation would train specialist policies per niche via gradient descent while retaining ENA's environment detection and Bayesian policy reuse.

Finally, wall-clock execution times reflect specific software implementations, measured on a single workstation for CartPole and on dedicated cloud instances per replicate for HalfCheetah. While environment steps and candidate steps represent intrinsic properties of the algorithms, wall-clock time also reflects implementation and hardware utilization efficiency; hence, it does not imply that ENA is inherently less computationally demanding across all execution platforms.

E. Limitations

- *Two benchmark families.* Both use related physics changes, with a single morphology in each. Visual, partially observable or structurally different tasks are untested.
- *Fixed task order.* Each domain uses one training sequence (Mars–Jupiter–Earth; Earth–Heavy-Rough–Mars-Slippery). Ordering effects are untested.
- *No recurrence during training.* Earlier environments return only at the frozen test. Repeated switches, short dwell times and recall during training are untested.
- *Test-time choice in the ablation.* Without a Hall of Fame, there are no gladiators to choose from. In that case, ENA-NoSkill and ENA-Plasticity play a random member of their final population in every test episode. The variants with an archive choose among gladiators by Bayesian policy reuse. Adding the Hall of Fame thus changes both what is kept and how the test policy is chosen, and the ablation does not separate the two.
- *Interaction in the ablation.* The ablation fixes the number of episodes, not of environment steps. A variant with Plasticity takes many times more steps than one without (Section V-A). The ablation shows what failure-driven search achieves with that interaction. It does not show that allocating effort by failure beats spreading the same effort evenly. Full search was run in a single replicate.
- *Resets.* The chirp requires resetting the environment at episode boundaries, to play its 64-step excitation from nominal initial states. Candidate evaluation needs more than a reset: a resettable copy of the current physics. Both are natural in simulation, digital twins and automated test rigs. They constrain continuous, non-resettable physical systems.

VII. Conclusion and Future Work

This work investigated whether a small set of modular skills can transform the solutions generated by an evolving population into a reusable memory that mitigates catastrophic forgetting. To address this question, the Evolutionary Neuroplastic Agent (ENA) was introduced. Across the evaluated CartPole sequences, the empirical ablation supports this architecture. Only variants with failure-driven search master the more demanding environments, requiring increased environment interaction. Among these variants, only the architecture that indexes its archive by detected physical regimes preserves acquired competencies across switches. Introducing the archive also alters test-time policy selection, an effect that the current ablation does not decouple. Policy retrieval via Bayesian policy reuse proved significantly more reliable than relying solely on change detection. The memory is constructed online without external supervision, using detection mechanisms tailored to each domain. Whether dynamically modulating search effort based on failure outperforms uniform effort allocation remains an open question. Single-policy learners trained on identical sequences suffer catastrophic forgetting of previously acquired behaviors. Conversely, they highlight regimes where ENA is less competitive: on CartPole, adaptation is slower than PPO, and on HalfCheetah, asymptotic returns are lower than those of SAC. Both limitations align with known properties of evolutionary search, which exhibits high sample complexity and scales unfavorably with parameter count [19], [47].

The most direct next step is a hybrid of ENA with gradient-based learners. Evolution and gradient descent can be combined in multiple configurations, offering several promising avenues. Gradient learners such as SAC or PPO could serve as local optimizers within each niche to accelerate adaptation and achieve higher asymptotic returns. ENA's environment detection, Plasticity, and physics-indexed Hall of Fame would preserve niche-specific competencies across dynamic switches. Alternatively, the architectures could partition the policy network—with gradient descent optimizing high-dimensional feature representations and evolutionary search tuning low-dimensional control layers—or alternate sequentially. The evolutionary population could also supply diverse rollouts to off-policy replay buffers, as in evolutionary reinforcement learning [19]. A hybrid architecture would furthermore provide a direct comparison against continual-RL frameworks designed for contextual non-stationarity: OWL [5], RL-CD [6], MBCD [7], SWOKS [8], and TeLAPA [24].

The modular skills may also transfer directly to those frameworks. OWL and TeLAPA depend on explicit task boundaries; their authors identify automated task boundary detection during training [5] and handling ambiguous transitions [24] as primary challenges. ITE initiates adaptation based on a predefined performance threshold and maintains no memory across sequential conditions [23]. ENA's environment detection identifies transitions and recognizes recurring regimes directly from unassisted observations, providing capabilities that existing methods either assume or lack. Furthermore, a physics-indexed archive could preserve adapted policies for deployment when previously encountered conditions recur.

Several further directions remain open. Repeated recurrence during training and randomized task orders would test the

memory beyond the evidence here. For systems that cannot be reset, the probe could be superimposed on the agent's own actions as a small dither. The contributions of the belief's two sources of evidence, the dynamics likelihoods and the test returns, remain to be separated. Specialist selection could also be learned, as a contextual bandit over the detector's evidence. Finally, the population's evaluations are independent and parallelize naturally on accelerators, which would allow larger populations and longer non-stationary streams.

Code Availability

The full implementation of the Evolutionary Neuroplastic Agent, the experiment runners, the baselines, the LaTeX source of this paper and the scripts that generate its tables and figures are publicly available:

- **GitHub Repository:**
  https://github.com/blasvicco/ena

The results reported here were produced with the tagged release [TODO: release tag and archived DOI (e.g. Zenodo) of the revision's code and experiment outputs], which fixes the code, configuration and raw replicate-level outputs behind every number.

References

[1] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," in *Psychology of Learning and Motivation*. Academic Press, 1989, vol. 24, pp. 109–165.

[2] K. Khetarpal, M. Riemer, I. Rish, and D. Precup, "Towards continual reinforcement learning: A review and perspectives," *Journal of Artificial Intelligence Research*, vol. 75, pp. 1401–1476, 2022.

[3] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[4] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[5] S. Kessler, J. Parker-Holder, P. Ball, S. Zohren, and S. J. Roberts, "Same state, different task: Continual reinforcement learning without interference," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 7143–7151.

[6] B. C. da Silva, E. W. Basso, A. L. C. Bazzan, and P. M. Engel, "Dealing with non-stationary environments using context detection," in *Proceedings of the 23rd International Conference on Machine Learning*, 2006, pp. 217–224.

[7] L. N. Alegre, A. L. C. Bazzan, and B. C. da Silva, "Minimum-delay adaptation in non-stationary reinforcement learning via online high-confidence change-point detection," in *Proceedings of the 20th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2021, pp. 97–105.

[8] J. Dick, S. Nath, C. Peridis, E. Benjamin, S. Kolouri, and A. Soltoggio, "Statistical context detection for deep lifelong reinforcement learning," in *Proceedings of the 3rd Conference on Lifelong Learning Agents (CoLLAs)*, 2024.

[9] E. Nisioti, A. Cossu, K. Korte, and S. Risi, "Continual reinforcement learning with neuroevolution," *arXiv preprint arXiv:2610.01583*, 2026.

[10] B. Rosman, M. Hawasly, and S. Ramamoorthy, "Bayesian policy reuse," *Machine Learning*, vol. 104, no. 1, pp. 99–127, 2016.

[11] A. G. Barto, R. S. Sutton, and C. W. Anderson, "Neuronlike adaptive elements that can solve difficult learning control problems," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983.

[12] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012, pp. 5026–5033.

[13] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[15] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 80, 2018, pp. 1861–1870.

[16] K. O. Stanley, J. Clune, J. Lehman, and R. Miikkulainen, "Designing neural networks through neuroevolution," *Nature Machine Intelligence*, vol. 1, no. 1, pp. 24–35, 2019.

[17] T. Salimans, J. Ho, X. Chen, S. Sidor, and I. Sutskever, "Evolution strategies as a scalable alternative to reinforcement learning," *arXiv preprint arXiv:1703.03864*, 2017.

[18] F. P. Such, V. Madhavan, E. Conti, J. Lehman, K. O. Stanley, and J. Clune, "Deep neuroevolution: Genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning," *arXiv preprint arXiv:1712.06567*, 2017.

[19] S. Khadka and K. Tumer, "Evolution-guided policy gradient in reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 31, 2018, pp. 1188–1200.

[20] A. Pourchot and O. Sigaud, "CEM-RL: Combining evolutionary and gradient-based methods for policy search," in *International Conference on Learning Representations (ICLR)*, 2019.

[21] Y. Jin and J. Branke, "Evolutionary optimization in uncertain environments—a survey," *IEEE Transactions on Evolutionary Computation*, vol. 9, no. 3, pp. 303–317, 2005.

[22] J.-B. Mouret and J. Clune, "Illuminating search spaces by mapping elites," *arXiv preprint arXiv:1504.04909*, 2015.

[23] A. Cully, J. Clune, D. Tarapore, and J.-B. Mouret, "Robots that can adapt like animals," *Nature*, vol. 521, no. 7553, pp. 503–507, 2015.

[24] L. Lillo and N. Cheney, "Beyond single-model optimization: Preserving plasticity in continual reinforcement learning," in *Proceedings of the 5th Conference on Lifelong Learning Agents (CoLLAs)*, 2026, arXiv:2604.15414.

[25] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1–2, pp. 100–115, 1954.

[26] R. P. Adams and D. J. C. MacKay, "Bayesian online changepoint detection," *arXiv preprint arXiv:0710.3742*, 2007.

[27] T. Halme, H. V. Poor, and V. Koivunen, "Adaptive sequential change detection using mixtures of predictive distributions," *arXiv preprint arXiv:2606.05072*, 2026.

[28] R. S. Galhardo, P. J. Hastings, and S. M. Rosenberg, "Mutation as a stress response and the regulation of evolvability," *Critical Reviews in Biochemistry and Molecular Biology*, vol. 42, no. 5, pp. 399–435, 2007.

[29] H. G. Cobb, "An investigation into the use of hypermutation as an adaptive operator in genetic algorithms having continuous, time-dependent nonstationary environments," Naval Research Laboratory, Washington, DC, Tech. Rep. AIC-90-001, 1990.

[30] A. E. Eiben, R. Hinterding, and Z. Michalewicz, "Parameter control in evolutionary algorithms," *IEEE Transactions on Evolutionary Computation*, vol. 3, no. 2, pp. 124–141, 1999.

[31] J. J. Grefenstette, "Genetic algorithms for changing environments," in *Parallel Problem Solving from Nature 2*, R. Männer and B. Manderick, Eds. North-Holland, 1992, pp. 137–144.

[32] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly, "Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory," *Psychological Review*, vol. 102, no. 3, pp. 419–457, 1995.

[33] C. D. Rosin and R. K. Belew, "New methods for competitive coevolution," *Evolutionary Computation*, vol. 5, no. 1, pp. 1–29, 1997.

[34] J. Branke, "Memory enhanced evolutionary algorithms for changing optimization problems," in *Proceedings of the 1999 Congress on Evolutionary Computation (CEC)*, vol. 3, 1999, pp. 1875–1882.

[35] E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.

[36] M. Herbster and M. K. Warmuth, "Tracking the best expert," *Machine Learning*, vol. 32, no. 2, pp. 151–178, 1998.
[37] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1999.
[38] W. Zhou, L. Pinto, and A. Gupta, "Environment probing interaction policies," in *International Conference on Learning Representations*, 2019.
[39] R. R. Coifman and M. V. Wickerhauser, "Entropy-based algorithms for best basis selection," *IEEE Transactions on Information Theory*, vol. 38, no. 2, pp. 713–718, 1992.
[40] S. G. Mallat, "A theory for multiresolution signal decomposition: The wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 11, no. 7, pp. 674–693, 1989.
[41] T. Kijewski and A. Kareem, "Wavelet transforms for system identification in civil engineering," *Computer-Aided Civil and Infrastructure Engineering*, vol. 18, no. 5, pp. 339–355, 2003.
[42] Z. K. Peng and F. L. Chu, "Application of the wavelet transform in machine condition monitoring and fault diagnostics: A review with bibliography," *Mechanical Systems and Signal Processing*, vol. 18, no. 2, pp. 199–221, 2004.
[43] G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt, and A. O'Leary, "PyWavelets: A Python package for wavelet analysis," *Journal of Open Source Software*, vol. 4, no. 36, p. 1237, 2019.
[44] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.
[45] P. Christodoulou, "Soft actor-critic for discrete action settings," *arXiv preprint arXiv:1910.07207*, 2019.
[46] I. Daubechies, "Orthonormal bases of compactly supported wavelets," *Communications on Pure and Applied Mathematics*, vol. 41, no. 7, pp. 909–996, 1988.
[47] A. Vemula, W. Sun, and J. A. Bagnell, "Contrasting exploration in parameter and action space: A zeroth-order optimization perspective," in *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 89, 2019, pp. 2926–2935.
[48] J. Lehman, J. Chen, J. Clune, and K. O. Stanley, "Safe mutations for deep and recurrent neural networks through output gradients," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2018.
[49] A. Raffin, "RL baselines3 zoo," https://github.com/DLR-RM/rl-baselines3-zoo, 2020.

APPENDIX A
INFORMATION AVAILABLE TO EACH AGENT

A comparison across agent families is only fair if no agent learns more about the environment than the others. This appendix lists everything each agent receives.

A. The interface

Every agent, ENA and the baselines alike, interacts with an environment only through the Gymnasium interface. A reset returns an initial observation. A step returns the next observation, the reward, and whether the episode terminated or was truncated by the time limit. In CartPole, the observation is the cart's position and velocity, and the pole's angle and angular velocity. In HalfCheetah, it is the torso's height and angle, the joint angles, and all their velocities, without the forward position.

The physical parameters that distinguish the environments are never part of the observation. In CartPole, these are gravity, the pole's mass and the pole's length. In HalfCheetah, they are gravity, friction, wind and the presence of air. No agent is told when the environment changes, which environment it is in, or how many environments there are. The environment names in the figures and tables are taken from the experiment logs, and are used only for analysis. ENA's niches are its own numbering.

B. Task constants

ENA reads two constants from each task's Gymnasium registration: the episode time limit, and the solved threshold R_{solved}. They are 500 steps and 475 in CartPole, and 1,000 steps and 4,800 in HalfCheetah. The time limit rescales a truncated candidate return to a full-episode equivalent (4). HalfCheetah episodes never end early, and its candidates are evaluated for at most 200 steps, so every HalfCheetah candidate's return is rescaled. The solved threshold is the utility scale of the specialist selection (8). The selection therefore depends on this registered constant, even on HalfCheetah, where no agent reaches it. There, the threshold sets how much one return moves the belief, since the return noise of (8) is 0.2 times the threshold in return units. Until 10 returns of the current regime have been seen, it is also the scale against which returns are normalized for Trust. After that, Trust normalizes against the percentiles of recent returns (3). Both constants are public properties of the task, not of the physics. The baselines meet the same time limit through the truncation signal.

C. Environment detection

The detector's input is the sequence of transitions (s_t, a_t, s_{t+1}): the observation before acting, the action taken, and the observation after. This is exactly what the agent sees. The detector does not read the reward, the physical parameters or the environment's name. A change of physics changes how the observed velocities evolve, so the current niche's dynamics model (5) predicts the new transitions worse. In the ablation, changes between the training environments are detected almost immediately and almost without error (Table XVI).

At test, the detector is frozen and creates no niches. Physics close to a known niche are recognized as that niche. The detector does not need to name an environment, only to find the closest physics it knows. Neptune's physics, for instance, are recognized as Earth's in 30 of the 30 replicates. When no niche explains the transitions within the margin m, the episode is left unrecognized, and no niche is assigned. The selection (8) still weighs every niche by how well its dynamics model explains the episode, so it falls back on the niches whose models explain the episode best.

Recognition is weaker in two cases:

- *False alarms.* In a test phase, every confirmed change except the first one after a switch is a false alarm. In physics near the margin m, the detector can alternate between recognizing a niche and leaving the physics unrecognized. Each confirmation resets the selection's belief to uniform.
- *Fragmentation.* In some replicates, one training environment is split over two niches.

Table XVI gives these numbers for the two ablation variants with environment detection. For each phase, it lists the share

of unrecognized episodes, the confirmed changes and the episode of the first one. For each training environment, it lists the Hall of Fame niches. Every phase except the run's first follows a real switch. At most one confirmed change per phase is therefore a detection, and the rest are false alarms. The first training phase, Mars, starts with no niche at all. The detector is also reset when the test begins, so the test's Mars phase starts without a current niche, like the run's first. The Brown-dwarf phase is mostly left unrecognized. After it, the detector recognizes Earth by identification, without confirming a change, and Earth shows zero confirmed changes at test.

TABLE XVI
ENVIRONMENT RECOGNITION IN THE SKILL ABLATION (30 REPLICATES)

Agent	Phase	Env.	Unrec. (%)	Changes	First (ep.)	Niches
ENA-HoF+EnvDetection	Training	Mars	0.4 ± 0.4	1.3 ± 1.9	243.0 ± 138.7	1.6 ± 0.6
ENA-HoF+EnvDetection	Training	Jupiter	0.2 ± 0.0	1.1 ± 0.3	1.0 ± 1.0	1.1 ± 0.3
ENA-HoF+EnvDetection	Training	Earth	0.1 ± 0.2	1.2 ± 0.4	0.5 ± 0.7	1.2 ± 0.4
ENA-HoF+EnvDetection	Test	Mars	9.8 ± 24.3	2.0 ± 10.6	6.3 ± 4.2	–
ENA-HoF+EnvDetection	Test	Neptune	1.9 ± 5.7	2.4 ± 5.8	0.2 ± 0.5	–
ENA-HoF+EnvDetection	Test	Jupiter	0.8 ± 0.8	1.0 ± 0.0	0.2 ± 0.4	–
ENA-HoF+EnvDetection	Test	Brown	96.5 ± 11.4	6.8 ± 11.3	0.2 ± 0.5	–
ENA-HoF+EnvDetection	Test	Earth	1.0 ± 0.9	0.0 ± 0.0	–	–
ENA-Combined	Training	Mars	0.3 ± 0.3	1.3 ± 2.5	202.4 ± 174.3	1.4 ± 0.5
ENA-Combined	Training	Jupiter	0.2 ± 0.1	1.1 ± 0.4	0.8 ± 0.9	1.1 ± 0.4
ENA-Combined	Training	Earth	0.0 ± 0.0	1.3 ± 0.5	0.0 ± 0.0	1.3 ± 0.5
ENA-Combined	Test	Mars	1.1 ± 1.1	0.8 ± 1.7	29.5 ± 68.8	–
ENA-Combined	Test	Neptune	0.8 ± 1.1	1.2 ± 1.1	0.3 ± 0.5	–
ENA-Combined	Test	Jupiter	0.6 ± 0.3	1.0 ± 0.0	0.2 ± 0.4	–
ENA-Combined	Test	Brown	92.6 ± 12.6	16.4 ± 18.8	0.0 ± 0.0	–
ENA-Combined	Test	Earth	0.4 ± 0.2	0.0 ± 0.0	–	–

On HalfCheetah, ENA detects with its chirp (Section III-H5). The chirp's input is the agent's own fixed action sequence, and the observations it produces after a reset of the environment. These are the same observations the agent sees, so the chirp reads no reward, physical parameter or environment name either. Its steps count in ENA's step budget. The environment's random stream is restored after it, so every agent's episodes start from the same states. Table XVII gives the same numbers for ENA-Combined in the HalfCheetah study.

TABLE XVII
ENVIRONMENT RECOGNITION ON HALFCHEETAH (20 REPLICATES)

Agent	Phase	Env.	Unrec. (%)	Changes	First (ep.)	Niches
ENA-Combined	Training	Earth	0.0 ± 0.0	0.0 ± 0.0	–	1.0 ± 0.0
ENA-Combined	Training	Heavy	1.0 ± 0.7	2.1 ± 1.0	0.0 ± 0.0	1.4 ± 0.6
ENA-Combined	Training	Mars	1.3 ± 0.6	1.8 ± 0.8	0.7 ± 2.3	1.3 ± 0.6
ENA-Combined	Test	Earth	0.0 ± 0.0	0.0 ± 0.0	–	–
ENA-Combined	Test	Ice	52.2 ± 13.0	6.3 ± 1.5	1.4 ± 0.5	–
ENA-Combined	Test	Heavy	1.1 ± 1.0	0.5 ± 0.5	6.2 ± 14.7	–
ENA-Combined	Test	High	2.5 ± 0.9	1.0 ± 0.0	0.8 ± 0.7	–
ENA-Combined	Test	Mars	4.3 ± 1.5	1.2 ± 0.4	0.1 ± 0.2	–

D. Candidate evaluation

ENA evaluates candidate specialists in fresh instances of the current environment. The experiment's own environment factory builds them from the current environment's physical parameters. ENA passes these parameters to the factory, but uses none of them for a decision. Every candidate step counts in ENA's interaction budget. A candidate evaluation is thus equivalent to an extra episode in the current environment, with the same reset that any episodic agent has. The baselines could spend the same budget in parallel copies of the environment, as vectorized training does. What ENA does assume is a resettable copy of the current physics, in which it can run episodes other than the one being played. Such a copy is free in simulation or with a digital twin. On a physical system, it would cost real time, if it can be had at all.

E. Evaluation and test

The first-solve checks (Section IV-B2) run on a separate copy of the current environment, for every agent alike. Their steps do not count as training. At test, every agent is frozen, and Section III-I lists what still adapts in ENA. Every agent of a replicate meets the same seeded environments.

APPENDIX B
HYPERPARAMETERS

All settings are read directly from the code the studies ran, by `src/tools/paper_config.py`. Each value is an agent's class default, unless its study script overrides it. The values here therefore cannot drift from the code.

Table XVIII gives the settings of ENA-Combined, as built by `cartpole_ena_ablation.py`. All six ablation variants share them, with the relevant skills switched on or off. Table XIX lists only the settings that differ on HalfCheetah, set in `half_cheetah.py`.

On HalfCheetah, PPO and SAC take the settings of RL Baselines3 Zoo [49] (Table XXIII). Both have a separate actor and critic. PPO normalizes observations, and scales rewards by the standard deviation of the return. Two settings deviate from the zoo. PPO's learning rate is not annealed, because annealing it to zero would freeze PPO in its last environment. SAC applies its gradient steps in groups of 20 every 20 environment steps, for speed, which preserves the zoo's ratio of one update per step.

TABLE XVIII
ENA HYPERPARAMETERS (CARTPOLE)

Parameter	Value	Parameter	Value
Network (hidden)	4×2 ReLU	Population size N	100
Eval depth $s_{\min}$ / $s_{\max}$	10 / 500	Min. candidates	2
Return history	100	Exploration ε	0.2
Trust percentiles	5 / 95	Trust clip	$[-2, 5]$
Trust blend threshold	0.1	Signal warm-up (returns)	10
Signal percentile	90th	Plasticity controller	linear ramp
Mutation rate p_m	0.05	Mutation noise σ_m / floor	0.1 / 0.01
Crossover parent share	0.5	Elite (% of pop.)	20
Mutants (% of pop.)	32	Crossovers (% of pop.)	32
HoF K (max per niche)	5	HoF reinject ratio	0.5
HoF similarity threshold	0.95	HoF gate z / relief	1.5 / 0.5
HoF gate σ_T floor	0.1	HoF gate: sufficient Trust	0.5
Detection tolerance τ	3	Detection margin m	2
Detection recent window	500	Min. samples to fit	100
Verification samples	50	CuSum false-alarm run γ	10^8
BPR return noise	0.2	BPR switch rate	0.05
BPR prior strength	1	BPR prior own / other niche	1 / 0.5

TABLE XIX
ENA HYPERPARAMETERS THAT DIFFER ON HALFCHEETAH

Parameter	Value	Parameter	Value
Network (hidden)	32×16 tanh, normc	Eval depth $s_{\max}$	200
Mutation rate p_m	0.25	HoF K (max per niche)	10
Hall of Fame	olympic	Olympiad interval (steps)	200,000
Olympiad episodes	3	Qualifier seats per niche	2
Environment detection	chirp	Chirp steps	64
Chirp excitation	wavelet train (Mexican hat)	Chirp transform	wavelet packet
Chirp wavelet	Haar	Chirp level / bands	3 / 8
Chirp window (episodes)	4	Chirp min chirps	8
Chirp confirm margin	3		

TABLE XX
DQN HYPERPARAMETERS (CARTPOLE)

Parameter	Value	Parameter	Value
Learning rate	3×10^{-4}	Discount γ	0.99
Batch size	64	Replay buffer	50,000
ε start / end	1 / 0.01	ε decay steps	10,000
Target network update (steps)	1,000	Learning starts (steps)	1,000
Update every (steps)	4	Network	64×64 ReLU

TABLE XXI
PPO HYPERPARAMETERS (CARTPOLE)

Parameter	Value	Parameter	Value
Learning rate	3×10^{-4}	Learning rate anneal	no
Steps per update	2,048	Batch size	64
Epochs per update	10	Discount γ	0.99
GAE λ	0.95	Clip ratio	0.2
Entropy coefficient	0.01	Value coefficient	0.5
Max grad norm	0.5	Actor / critic	64×64 tanh
Initialization	orthogonal	Normalization	none

TABLE XXII
SAC-DISCRETE HYPERPARAMETERS (CARTPOLE)

Parameter	Value	Parameter	Value
Learning rate	3×10^{-4}	Discount γ	0.99
Batch size	64	Replay buffer	100,000
Learning starts (steps)	1,000	Update every (steps)	4
Target update τ	0.005	Target entropy	$0.5 \times \log \lvert\mathcal{A}\rvert$
Actor / twin critics	64×64 ReLU		

TABLE XXIII
PPO AND SAC HYPERPARAMETERS (HALFCHEETAH)

Parameter	PPO	SAC
Actor / critic	256×256 ReLU	256×256 ReLU
Learning rate	2.0633×10^{-5}	3×10^{-4}
Discount γ	0.98	0.99
Batch size	64	256
Steps per update / buffer	512	1,000,000
Epochs / gradient steps	20	20 per 20 steps
GAE λ	0.92	—
Clip ratio	0.1	—
Entropy coef. / temperature	4.01762×10^{-4}	auto ($-\dim \mathcal{A}$)
Value coef. / target update τ	0.58096	0.005
Max grad norm	0.8	—
Initial log std	-2	—
Normalization	obs., reward	none
Learning starts (steps)	—	10,000

APPENDIX C
COMPUTE ENVIRONMENT

CartPole experiments (all studies except HalfCheetah)

All CartPole studies ran inside a Docker container (`hub/python/Dockerfile`) on the host machine:

- **Host:** Apple MacBook Pro, Apple M1 Pro chip, 8 cores (6 performance + 2 efficiency), 16 GB RAM.
- **Container OS:** Linux (ARM64, via Docker Desktop on macOS).
- **Cores:** the runner used 6 of the 8 cores, running up to 6 replicates in parallel, one process each. Within a replicate, the agents trained one after another, and ENA evaluated its candidates on one thread. Wall-clock times are therefore measured under this shared load.
- **Python:** 3.11 (Docker image `python:3.11-slim`).
- **Key libraries (pinned):** TensorFlow 2.21.0, Gymnasium[MuJoCo] 1.3.0, MuJoCo 3.13.0, NumPy 2.4.6, Keras 3.15.1, SciPy 1.17.1, PyWavelets 1.8.0. Full list in `app/requirements.txt`.
- **Seeds** (Section IV): 1000–1029 for the ablation and for the Plasticity-controller and selection studies; 1000–1049 for the comparison with single-policy learners.

HalfCheetah experiment (cloud run)

The HalfCheetah comparison ran in AWS (eu-west-1), on one **c8g.xlarge** instance per replicate, 20 instances in all:[1]

- **Instance:** AWS c8g.xlarge, 4 vCPU (AWS Graviton4, ARM64).
- **Container:** Linux aarch64 (ARM64 Docker image, `hub/cloud/Dockerfile`), built from the same base image and pinned `requirements.txt` as the CartPole environment.
- **Hardware and threads, as each replicate recorded them:** Linux aarch64, 4 cores; TensorFlow threads all; ENA threads 2.
- **Seeds** (Section IV): 1000–1019.

APPENDIX D
EXTENDED EVALUATION AND TRAINING DYNAMICS

This appendix shows the episode-by-episode test curves of both domains, and HalfCheetah's training curves. The numbers they summarize are in Section V.

A. Frozen Test on CartPole

Fig. 5 shows the frozen test of the CartPole comparison. Each agent plays 200 episodes in each of five environments: Mars-Light → Neptune → Jupiter-Heavy → Brown-dwarf → Earth-Standard. The line is the mean return over the 50 replicates, and the band the middle 50% of replicates. On Jupiter-Heavy, the environment trained just before the last, the single-policy learners' returns stay low for the whole phase. ENA-Combined's return there stays close to the success threshold. On Brown-dwarf, ENA-Combined's mean return rises during the phase. The factors driving this increase were not investigated.

[1] Instance type defined in `infra/src/qos/qos.json`.

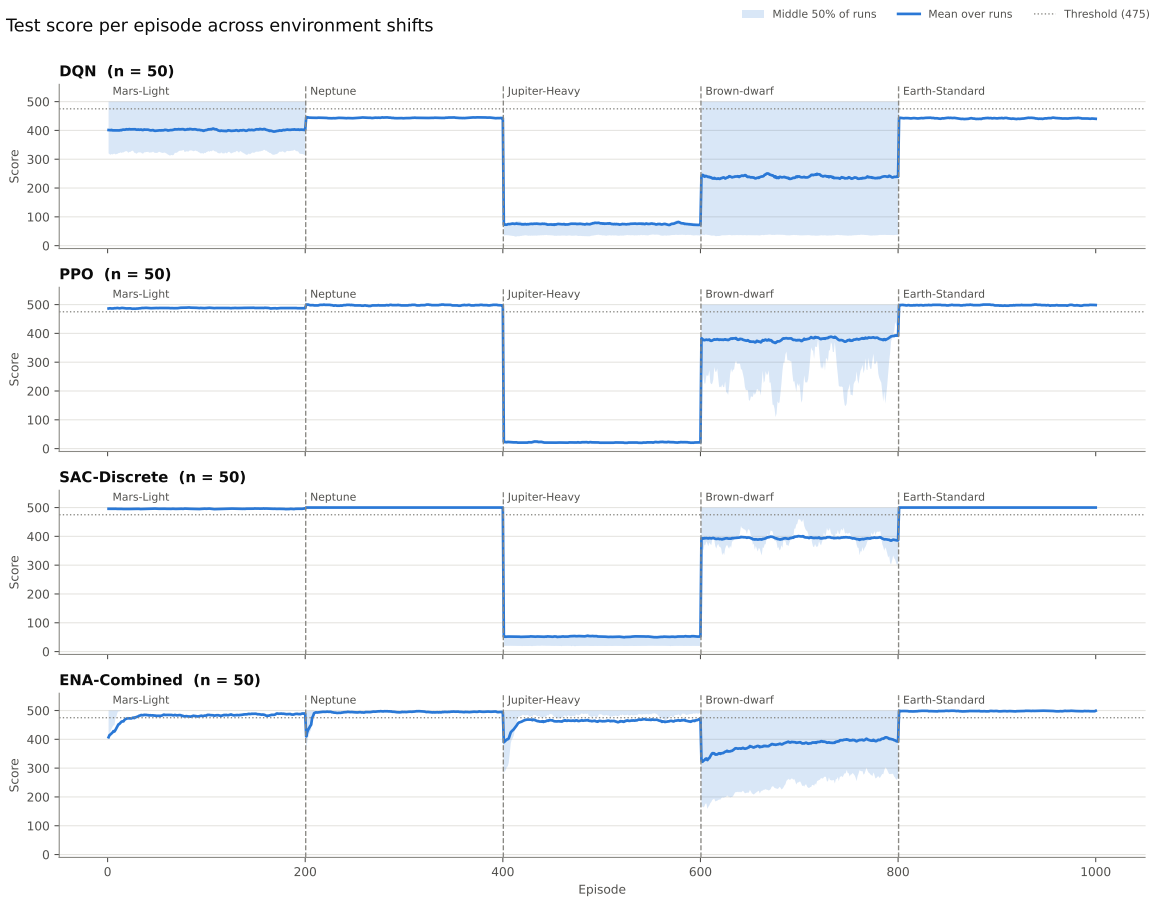


Fig. 5. Frozen test on CartPole: mean return per episode across 50 replicates (line) and the middle 50% of replicates (band). Dashed lines mark environment switches; Neptune and Brown-dwarf were never seen in training.

B. Frozen Test on HalfCheetah

Fig. 6 shows the frozen test on HalfCheetah, over the 20 replicates. Each agent plays 50 episodes in each of five environments: Earth-Standard → Ice-Floor → Heavy-Rough → High-Wind → Mars-Slippery. On Heavy-Rough, the environment trained just before the last, PPO's and SAC's returns drop. ENA-Combined's return there is its highest. Among the unseen environments, SAC's return on Ice-Floor is far above the other agents', and ENA-Combined's lowest return is on High-Wind.

C. Training Dynamics on HalfCheetah

Fig. 7 shows the training return of the three agents against environment steps, 1,000,000 per training environment. The returns are binned and averaged over the 20 replicates. SAC's return climbs within each phase, drops at each switch, and climbs again. It ends its three phases at 8,635, 7,544 and 9,405. ENA-Combined's training return stays far lower. It ends its phases at 1,805, 2,133 and 1,143.

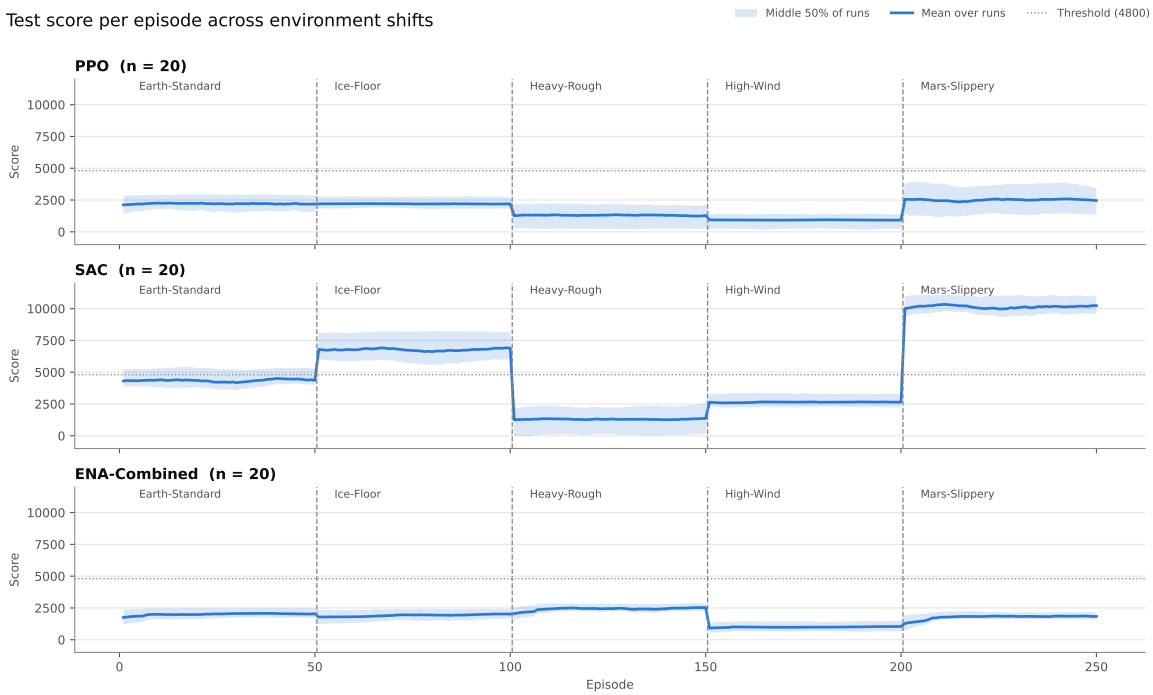


Fig. 6. Frozen test on HalfCheetah: mean return per episode across 20 replicates (line) and the middle 50% of replicates (band). Dashed vertical lines mark environment switches; Ice-Floor and High-Wind were never seen in training.

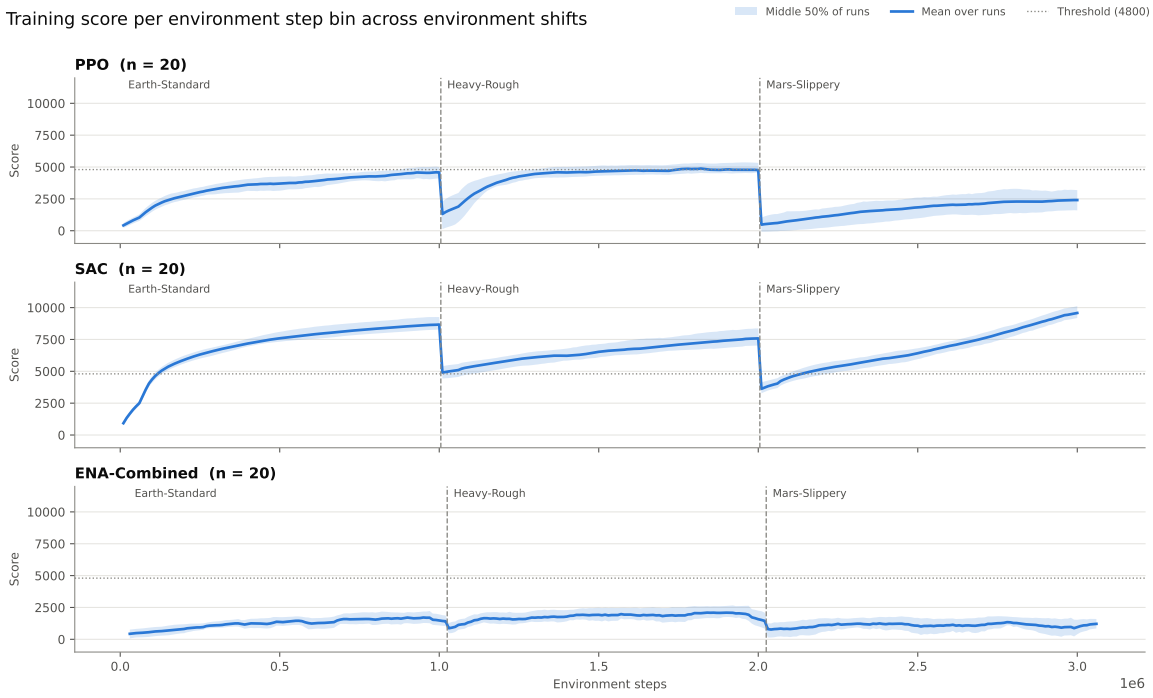


Fig. 7. Training return on HalfCheetah against environment steps (20 replicates; line is the mean, band the middle 50% of replicates). Dashed vertical lines mark the switches between the three training environments (Earth-Standard → Heavy-Rough → Mars-Slippery).